

Ecological Analyses in Julia

An introduction to ecological data science with Julia

Eduardo Santos

2026-10-03

Table of contents

Preface	3
Scope of this edition	3
How to use this book	3
Relationship to <i>Análises Ecológicas no R</i>	4
Relationship to <i>ecoR</i>	4
Bilingual edition	5
License	5
 I Foundations	 6
1 Introduction	7
1.1 Why Julia?	7
1.2 The honest cost	7
1.3 Compilation latency	8
1.4 Structure of the book	8
Exercises	9
 2 From question to model	 10
2.1 Why a chapter about questions in a book about code	10
2.2 Multiple working hypotheses	10
2.3 Design and dependence structure	10
2.4 Computation-limited questions	11
Further reading	11
Exercises	11
 3 Prerequisites	 12
3.1 Step 1: open a terminal	12
3.2 Windows	12
3.3 macOS	12
3.4 Linux	13
3.5 Step 2: install Julia	13
3.6 Windows	13
3.7 macOS and Linux	13
3.8 Step 3: download the book folder	13
3.9 Windows	14
3.10 macOS	14
3.11 Linux	14
3.12 Step 4: install VS Code	14
3.13 Step 5: open the book folder	15
3.14 Step 6: install the book's packages	15
3.15 Step 7: check that everything works	16
3.16 Step 8: a first script	16
3.17 Troubleshooting	17

3.18	Under the hood: project environments	18
3.19	Under the hood: Quarto	19
	Challenges	20
4	Introduction to Julia	21
4.1	Vectors and indexing	21
4.2	Broadcasting: the dot	23
4.3	Functions	24
4.4	Multiple dispatch	25
4.5	Missing data	26
4.6	Chaining	27
4.7	Control flow	28
4.8	Naming conventions	29
	Challenges	29
II	Wrangling and exploring data	31
5	Data wrangling with Tidier	32
5.1	The data	32
5.2	@chain: the pipe	33
5.3	The five verbs	34
5.3.1	@filter — pick rows	34
5.3.2	@select — pick columns	35
5.3.3	@mutate — create columns	36
5.3.4	@group_by + @summarize — aggregate	37
5.3.5	@arrange — sort	38
5.4	Missing data	38
5.5	Long and wide formats	40
5.6	Joins	40
5.7	DataFrames.jl underneath	41
	Challenges	42
6	Data visualization	43
6.1	TidierPlots.jl: the ggplot grammar	44
6.2	AlgebraOfGraphics.jl: an algebraic grammar	45
6.3	Facets	47
6.4	Distributions	49
6.5	Plain Makie.jl: full control	50
6.6	Saving figures	52
	Challenges	53
7	Exploratory data analysis	54
7.1	First contact: reading the file	55
7.2	Missing, unknown, and inconsistent	57
7.3	Structure: what is a replicate?	59
7.4	Distributions	61
7.5	Errors and outliers	63
7.6	Relationships and collinearity	65
7.7	Why the plots: the Anscombe quartet	69
	Challenges	70

III Modelling	71
8 Linear models	72
8.1 Hypothesis tests	72
8.2 Simple linear regression	74
8.3 Multiple regression	75
8.4 Categorical predictors and interactions	77
8.5 Model selection and comparison	78
8.6 Diagnostics	79
8.7 Mixed models	81
Challenges	83
9 Generalized Linear Models	84
9.1 Why GLMs in ecology	84
9.2 Poisson: species richness	85
9.3 Overdispersion	87
9.4 Offsets	89
9.5 Logistic regression	90
9.6 GLMMs	93
9.7 Gaps you will hit	94
9.7.1 The pragmatic escape hatch: <code>RCall.jl</code>	94
Challenges	95
IV Programming for ecology	96
10 Writing your own functions	97
10.1 From a calculation to a function	98
10.2 Arguments: defaults and keywords	99
10.3 Refusing bad input	100
10.4 Documenting a function	102
10.5 Your function, per group	103
10.6 Functions that take functions	105
10.7 Testing your functions	107
Challenges	107
11 Resampling and simulation	108
11.1 Randomness you can reproduce	109
11.2 Permutation tests	110
11.3 Permuting a regression	112
11.4 The bootstrap	113
11.5 How many ponds do I need?	116
11.6 A null model for species co-occurrence	118
Challenges	121
References	122
Change log	124
v1.0.0 (2026-10-02)	124

Preface

This book is an introduction to ecological data analysis using the [Julia](#) language. It began as an answer to a simple question: *how much of the ecologist’s toolkit already exists in Julia today, and how much still has to be built?*

The idea for this book was inspired by the wonderful project [Análises Ecológicas no R](#) (Da Silva et al. 2022) by Silva, Gonçalves-Souza, Paterno, Provete and Vancine — an open, Portuguese-language book that became the entry point for a generation of Brazilian ecologists. This volume deliberately follows its structure, chapter by chapter, so the two can be read side by side.

Scope of this edition

! What this book covers — and what it doesn’t yet

This first edition runs **from language basics through generalized linear models** (chapters 1–9), with a chapter on exploratory data analysis before the models, followed by two chapters on programming for ecology: writing your own functions and resampling and simulation (chapters 10–11). The community-ecology chapters of the original book (*Análises Ecológicas no R*) — multivariate analysis, rarefaction, richness estimators, taxonomic, phylogenetic and functional diversity, and geospatial data — **are not included**.

That is not an oversight. In R those chapters use packages such as `vegan`, `iNEXT`, `picante` and `FD`. Julia has no mature equivalents today. Writing those chapters honestly means writing the packages first, and that is a separate project (and not one for me). See Section [1.2](#) for a detailed assessment.

This is not a small slice. They cover what most of an ecological analysis actually consumes: importing and cleaning data, visualizing it, fitting linear and generalized linear models, and handling hierarchical structure through mixed models. At every one of those points the Julia ecosystem is solid: the packages exist, they are maintained, and they do the job very well. Where this book compares Julia to R, the comparisons are limited to what can be checked — syntax, feature coverage, behaviour demonstrable in the code — and not to performance claims I do not measure here.

How to use this book

All code is executable and runs in the Julia environment defined by `Project.toml`. To reproduce the book locally:

```
git clone https://github.com/esantos2ua/EcologicalAnalysesInJulia.git
cd EcologicalAnalysesInJulia
julia --project=. scripts/setup.jl
quarto render en
```

Throughout the text, boxes like this one translate from R's idiom into Julia's:

In R

`dplyr::filter(data, height > 10)` corresponds to `@filter(data, height > 10)` in `TidierData.jl`. The semantics are the same; the difference is that `@filter` is a macro rather than a function.

Relationship to *Análises Ecológicas no R*

This project is **inspired by** *Análises Ecológicas no R* (Da Silva et al. 2022), by Fernando Rodrigues da Silva, Thiago Gonçalves-Souza, Gustavo Brant Paterno, Diogo Borges Provete and Maurício Humberto Vancine, and owes it an explicit structural debt: this book's chapter sequence deliberately follows theirs, so the two can be read side by side.

What this book does **not** do is reproduce the original's content. The text, examples, dataset and exercises here are written from scratch for Julia. The authors of the original book have no involvement in this project, have not reviewed it, and do not endorse it; any errors here are mine.

Análises Ecológicas no R is freely available at <https://analises-ecologicas.com/> under a CC BY-NC 4.0 licence, and in print from Nupeea and Canal 6 (2022, 640 pp., ISBN 978-85-7917-564-0). If this book is useful to you, the original will probably be more so.

Relationship to *ecoR*

Chapters 7, 10 and 11 do not come from *Análises Ecológicas no R*. They are **inspired by** the exploratory-analysis, programming and resampling modules of **ecoR** (Oliveira 2026), the R course wiki of the Instituto de Biociências, Universidade de São Paulo, produced and maintained by Alexandre Adalardo de Oliveira, and based in part on material by João Luis Ferreira Batista and Paulo Inácio K. L. Prado. As with the original book, the text, examples and exercises here are written from scratch; the authors of *ecoR* have not reviewed this book. Each of the three chapters opens with links to the corresponding *ecoR* material.

ecoR is freely available at <https://ecor.ib.usp.br/> under a CC BY-SA 4.0 licence, with exercises graded automatically in [notaR](#).

Bilingual edition

This book exists in English and Portuguese, built from the same repository. The Portuguese version is at [/pt](#). The chapters are translations of one another, and the code is identical in both. I write mostly in Portuguese and use AI tools to translate the content into English, which I then review.

License

Text under [CC BY-NC-SA 4.0](#); code under the [MIT license](#). Contributions, corrections and bug reports are welcome in the repository.

Part I

Foundations

1 Introduction

1.1 Why Julia?

Julia was built to solve what its authors called the *two-language problem* (Bezanson et al. 2017): you prototype in a high-level, productive language (R, Python) and then, when the code is too slow, rewrite the bottleneck in C, C++ or Fortran. The result is that the library you use and the code you write are written in different languages — which makes the inside of a package opaque to its user.

In Julia, `vegan` would be written in Julia. That matters to an ecologist for three practical reasons:

1. **You can read the package you are using.** When `metaMDS` does something unexpected, the answer is in Fortran. In Julia, `@edit` drops you into readable source.
2. **Simulation stops being prohibitive.** Null models, bootstrapping, randomization — the bread and butter of community ecology — run in loops. Loops in Julia are fast.
3. **Packages compose.** Types defined in one package work with functions from another without glue code, because of multiple dispatch (Section 4.4).

1.2 The honest cost

These benefits are not free, and this book will not pretend otherwise. Let me lay out some of the gains and the costs.

What you gain	What you give up
Fast loops without vectorizing	The community-ecology ecosystem (<code>vegan</code> , <code>picante</code> , <code>FD</code>)
Readable package source <code>MixedModels.jl</code> , with <code>lme4</code> 's formula syntax	Generalized additive models (<code>mgcv</code>) DHARMA-style residual diagnostics
Composition through multiple dispatch	Critical mass of tutorials and Stack Overflow answers
One reproducible environment (<code>Project.toml</code> + <code>Manifest.toml</code>)	Compilation latency on first call

i How long this table stays true

The right-hand column is an assessment of the Julia ecosystem as of **August 2026**, made by checking the public package registries. Ecosystems change. Before concluding that something does not exist, check — and if you find a package that fills one of these gaps, this chapter is wrong and should be corrected. I will try to update it regularly, but I certainly won't manage to keep everything current.

i When *not* to use Julia

If your analysis is essentially `vegan::adonis2()` followed by `metaMDS()`, use R. This book stops at GLMs precisely because that is as far as the “use Julia” recommendation holds without caveats.

1.3 Compilation latency

Julia compiles each method the first time it is called with a given set of types. That is what makes the second (third, fourth, ...) call fast, and the first one slow.

```
sum_of_squares(v) = sum(x^2 for x in v)

v = randn(1_000_000)

# first call: includes compilation
@elapsed sum_of_squares(v)
```

0.030035531

```
# second call: execution only
@elapsed sum_of_squares(v)
```

0.000956414

The difference between the two is compilation time. In a real analysis session you pay that cost once. In a short script that runs a single time and exits, the compilation cost can have a noticeable impact. It is worth understanding the phenomenon a little before concluding that “Julia is a slow language”.

1.4 Structure of the book

- **Foundations** (Chapter 1 – Chapter 4): why Julia, how to frame the question, how to install, and the language itself.
- **Wrangling and visualization** (Chapter 5 – Chapter 6): `TidierData.jl` and the `Makie` family.
- **Modelling** (Chapter 8 – Chapter 9): `GLM.jl`, `MixedModels.jl`, `HypothesisTests.jl`.

Exercises

1. Install Julia via `juliaup` and reproduce the latency example above. How long does your first call take?
2. List three analyses you run often. For each, search [JuliaHub](#) for an equivalent package.

2 From question to model

! Status of this chapter

This chapter is not yet fully written. What follows is a **plan**, with its references. There is still a lot to develop here.

The plan is my own, not a summary of any other work. It does not reproduce the content of the corresponding chapter in Da Silva et al. (2022), which I did not consult while writing it.

2.1 Why a chapter about questions in a book about code

No amount of statistical sophistication rescues a badly posed question. The right order is: question → design → data → model → inference (one could write a whole book about that process). Tools like Julia act on the last two steps, and are of little use if the first two (question and sampling design) are not worked out with care.

This chapter exists to join the book's two halves: the question you ask determines the model structure that appears in Chapter 8 and Chapter 9.

2.2 Multiple working hypotheses

This section will be built on Chamberlin (1890), which argued for holding several rival hypotheses in play at once rather than defending a single one — an 1890 argument that remains the best available description of what separates a productive question from a defensive one.

To develop:

- Observation, pattern and process: the three levels, and why conflating them yields models that answer a different question than the one asked.
- Mechanistic versus phenomenological hypotheses, and what each licenses you to conclude.

2.3 Design and dependence structure

This section will be built on Hurlbert (1984), which named pseudoreplication and catalogued its forms, and on Zuur et al. (2009), whose treatment of dependence structure connects directly to the random effects used in this book.

To develop:

- Sampling unit, experimental unit, and why the distinction sets the denominator of your test.
- How design structure translates into (1 | group) terms — the direct link to 1.
- Effect size and power before collection, not after.

2.4 Computation-limited questions

A section specific to this book. Some ecological questions are limited by computation rather than by data: null models with many permutations, individual-based simulation, fitting mechanistic models by optimization. These benefit most from the language, for the reasons discussed in Section 1.1.

This section needs to be written with runnable examples, like the rest of the book — not with unverified performance claims.

Further reading

- Chamberlin (1890) on multiple working hypotheses.
- Hurlbert (1984) on pseudoreplication and experimental design.
- Zuur et al. (2009), chapter 1, on design and dependence structure.
- Da Silva et al. (2022), for the same material treated in R.

Exercises

1. Write your project's central question in one sentence, without statistical jargon.
2. Sketch the model it implies *before* looking at the data: what response, what predictors, what grouping structure?
3. Identify the experimental unit in your design. How many true replicates do you have — not how many measurements?

3 Prerequisites

This chapter gets your computer ready for the rest of the book. By the end you will have installed Julia and an editor, downloaded the book’s data and exercises, and run a first script that draws a plot.

Follow the steps in order. Where they differ between operating systems, pick your system’s tab. Plan for about an hour the first time — most of it is waiting while packages install — and keep an internet connection until Section 3.14 is done. If a step does not behave as described, see Section 3.17.

i What you will install

What	Why
Julia (through <code>juliaup</code>)	the language itself
VS Code and its Julia extension	where you write and run code
The book folder	data, exercises and the list of packages the book uses
The book’s packages	data frames, plots and models, at the exact versions the book was built with

You do not need git, R or Quarto.

3.1 Step 1: open a terminal

A terminal is a window where you type commands instead of clicking. You need it only for installing Julia; after that, everything happens inside VS Code.

3.2 Windows

Open the Start menu, type **PowerShell** and open *Windows PowerShell* (or *Terminal*, on Windows 11).

3.3 macOS

Press `Cmd+Space`, type **Terminal** and press Return.

3.4 Linux

Press `Ctrl+Alt+T` on most distributions, or look for *Terminal* in the applications menu.

To run a command, paste it into the terminal and press Enter. Pasting is `Ctrl+V` on Windows, `Cmd+V` on macOS and `Ctrl+Shift+V` in most Linux terminals.

3.5 Step 2: install Julia

Use **juliaup**, the official installer and version manager. Do not install Julia through a system package manager (`apt`, `brew`), which tends to ship outdated versions.

3.6 Windows

```
winget install --name Julia --id 9NJNWW8PVKMN -e -s msstore
```

If Windows asks you to accept the Microsoft Store terms, type `Y` and press Enter.

3.7 macOS and Linux

```
curl -fsSL https://install.julialang.org | sh
```

The installer asks whether to proceed with the default settings. Press Enter to accept them.

When it finishes, **close the terminal and open a new one** — the `julia` command only becomes available in new terminals. Then check the installation:

```
julia --version
```

It should print something like `julia version 1.12.6`. Any version from 1.10 upwards works with this book. Now type `julia` and press Enter to start Julia itself. The prompt changes to `julia>`; type `1 + 1`, press Enter, and Julia answers `2`. Leave with `exit()`.

Later, `juliaup update` upgrades Julia and `juliaup status` lists the installed versions.

3.8 Step 3: download the book folder

The book comes with a folder that holds the example data, the exercise checker and the list of packages the book needs.

1. Go to <https://github.com/esantos2ua/EcologicalAnalysesInJulia>.

2. Click the green **Code** button, then **Download ZIP**.
3. Unzip the file:

3.9 Windows

Right-click the downloaded file and choose **Extract All...** Work in the extracted folder, not inside the ZIP: files opened from inside a ZIP are read-only copies.

3.10 macOS

Double-click the downloaded file.

3.11 Linux

Right-click the file and choose *Extract Here*, or run `unzip` on it.

4. Move the extracted folder, `EcologicalAnalysesInJulia-main`, somewhere permanent such as your *Documents* folder. You can rename it, but avoid putting it inside *Downloads*, which some systems clean up automatically.

Inside you should see, among others, the folders `data` and `exercises` and the files `Project.toml` and `Manifest.toml`.

💡 If you already use git

Clone the repository instead of downloading the ZIP; `git pull` then brings in updates to the book:

```
git clone  
  ↪ https://github.com/esantos2ua/EcologicalAnalysesInJulia.git
```

3.12 Step 4: install VS Code

VS Code with the official Julia extension is the standard editor for Julia: it offers an integrated REPL (the console where code runs), a variable browser, a plot pane, a debugger and line-by-line evaluation — the functional equivalent of RStudio.

1. **Install VS Code** from <https://code.visualstudio.com> and open it.
2. **Install the Julia extension.** Open the Extensions panel (`Ctrl+Shift+X`; `Cmd+Shift+X` on macOS), search for *Julia* and install the extension published by *juliaup*. It finds the Julia installed by *juliaup* on its own.

If you prefer a fully open-source build, [VSCodium](#) is VS Code compiled from the same MIT-licensed source without Microsoft’s telemetry, and installs the same Julia extension from the [Open VSX](#) registry. [Antigravity](#), another editor derived from VS Code, also runs the same extension. Everything in this book applies to all three. Other options are **Pluto.jl** (reactive notebooks, excellent for teaching) and any editor with LSP support through `LanguageServer.jl`.

3.13 Step 5: open the book folder

1. **Open a folder, not a file.** In VS Code, use *File → Open Folder...* and choose the book folder from Section 3.8. If VS Code asks whether you trust the authors of the files, answer yes. The folder you open determines both where Julia looks for files and which packages it uses.
2. **Check the environment.** The status bar at the bottom of the window should read `Julia env:` followed by the name of the book folder. If it shows something like `Julia env: v1.12` instead, click that entry and choose the book folder.
3. **Start the REPL.** Open the Command Palette (`Ctrl+Shift+P`; `Cmd+Shift+P` on macOS) and run *Julia: Start REPL*, or press `Alt+J` then `Alt+O`. A panel opens at the bottom with the `julia>` prompt, already inside the book folder and its environment.

In R

Opening a folder in VS Code plays the role of opening an `.Rproj` in RStudio: it sets the working directory. Unlike RStudio, it also selects the package environment (Section 3.18), so the right package versions come along automatically.

3.14 Step 6: install the book’s packages

Click in the REPL panel and run:

```
using Pkg
Pkg.instantiate()
```

This reads `Manifest.toml`, downloads every package the book uses at the exact version listed there, and compiles them.

⚠ This takes a while

The first time, `Pkg.instantiate()` can take from several minutes to half an hour, depending on your computer and connection. Plotting and modelling packages are large, and Julia compiles them ahead of time so they run fast later. Progress lines keep appearing while it works; let it finish and do not close VS Code. You only do this once per computer.

3.15 Step 7: check that everything works

The book folder includes a checker that you will also use for the exercises from Chapter 4 on. Load it and ask it to check your setup:

```
include("exercises/check.jl")
check("setup")
```

If everything is in place you will see:

- ✓ Julia 1.12.6
- ✓ Working folder is the book folder
- ✓ Book environment is active
- ✓ Data file found: data/ponds.csv
- ✓ All 19 packages installed

Ready. Continue with the first script.

A line with `␣` says what is wrong and how to fix it; Section 3.17 has more detail.

3.16 Step 8: a first script

With the book folder open and the REPL running, create a file called `first-steps.jl` in the folder root (*File* → *New File*, or `Alt+J` then `Alt+N`) and paste:

```
## Load packages
using CSV, DataFrames, CairoMakie

## Where am I?
pwd()          # the folder you opened
readdir("data") # files inside data/

## Read the example data
ponds = CSV.read(joinpath("data", "ponds.csv"), DataFrame)
first(ponds, 5)
describe(ponds)

## Plot richness against pond area
fig = Figure()
ax = Axis(fig[1, 1]; xlabel = "Pond area (m²)", ylabel = "Species
↪ richness",
           xscale = log10)
scatter!(ax, ponds.area_m2, ponds.richness)
fig
```

Lines starting with `##` split the script into **cells**. Put the cursor in the first cell and press `Alt+Enter` to run it, then keep going cell by cell (`Alt+Shift+Enter` runs the cell and jumps to the next one). A few things to notice:

- **Loading packages is slow the first time** in each session — that is the compilation latency discussed in Section 1.3. Later cells run immediately.
- **Paths are relative to the folder you opened.** `pwd()` confirms it, which is why `joinpath("data", "ponds.csv")` works without typing the full path. `joinpath` inserts the right separator on every operating system.
- **describe is your first data check.** It shows that `canopy` has 7 missing values and that `area_m2` is strongly right-skewed (median around 149 m², maximum near 2000 m²) — which is why the plot uses a log scale.

The shortcuts you will use constantly:

Shortcut	Action
Shift+Enter	run the current line or selection, then move to the next
Ctrl+Enter	run the current line or selection, staying put
Alt+Enter	run the current cell (a block delimited by <code>##</code> lines)
Alt+J, Alt+R	restart the REPL
Ctrl+C in the REPL	interrupt a running computation

On macOS, Alt is the Option key; Ctrl stays Ctrl. Shortcuts and menu names were checked against version 1.249 of the Julia extension in September 2026.

Inspecting the environment. Click the Julia icon in the Activity Bar (the left-hand strip). The **Workspace** panel lists every variable in the session — `ponds`, `fig`, `ax` — with its type and size; expand an entry to look inside. For a data frame, hover over its name and click the *Open in VS Code* icon, or run `vscode.display(ponds)` in the REPL, to open it in a spreadsheet-like grid — the equivalent of RStudio’s `View()`.

Plots. Evaluating `fig` opens the figure in the **plot pane**, a VS Code tab. The **Plot Navigator** panel, below the Workspace, keeps the history of the session’s plots; with the plot pane focused, the arrow keys move through it. To save a figure to disk, use `save("richness-area.png", fig)`.

3.17 Troubleshooting

julia is not recognized / command not found. Close every terminal and open a new one; the installer only updates new terminals. If it still fails, run the installation command of Section 3.5 again.

Windows: winget is not recognized. `winget` comes with the *App Installer* app. Update *App Installer* from the Microsoft Store, or search for **Julia** in the Microsoft Store and install it from there — it is the same `juliaup` installer.

Linux: curl: command not found. Install `curl` with your distribution’s package manager (for example `sudo apt install curl` on Ubuntu), then repeat Section 3.5.

VS Code says it cannot find Julia. In a terminal, run `where.exe julia` (Windows) or `which julia` (macOS, Linux) and copy the path it prints. In VS Code, open the settings (`Ctrl+,`; `Cmd+,` on macOS), search for `julia.executablePath` and paste the path there. Then restart VS Code.

The status bar shows Julia env: v1.12 (or another version) instead of the book folder.

VS Code has the wrong folder open, or a single file. Use *File → Open Folder...* and choose the book folder itself — the one containing `Project.toml` — not a folder above or inside it. Then restart the REPL (`Alt+J`, `Alt+R`).

`include("exercises/check.jl")` fails with No such file or directory. The REPL is not running in the book folder. `pwd()` shows where it is. Open the book folder as above and restart the REPL.

`Pkg.instantiate()` seems frozen. It is probably compiling; this is the slow part. As long as new lines keep appearing every few minutes, wait. If it stopped with an error — for example because the connection dropped — run `Pkg.instantiate()` again; it picks up where it left off.

Package CairoMakie not found in current path (or another package). Either the book environment is not active (check the status bar) or Section 3.14 has not finished. `check("setup")` tells you which.

The first using CairoMakie in each session takes a minute. That is normal: Julia compiles code the first time it runs in a session (Section 1.3). Later commands are fast.

3.18 Under the hood: project environments

You have already used an environment without having to think about it: that is what the status bar in Section 3.13 shows. This section explains what it is — the concept that differs most from common practice in R.

In R, `install.packages()` installs into a global library shared by all your projects. In Julia, each project has its own environment, described by two files:

- **Project.toml** — the direct dependencies you declared.
- **Manifest.toml** — the full resolved dependency tree, with exact versions of everything.

Together they make an analysis reproducible by construction: anyone who has your project folder and runs `Pkg.instantiate()` gets exactly your versions — which is what you did in Section 3.14.

In R

`Project.toml` + `Manifest.toml` play the role of `renv.lock`, but they are part of the language rather than an add-on package. Activating an environment is the default operation, not the exception.

`Pkg.status()` lists the packages in the active environment:

```
using Pkg
Pkg.status()
```

```
Status `~/work/EcologicalAnalysesInJulia/EcologicalAnalysesInJulia/Project.toml`
^ [cbdf2221] AlgebraOfGraphics v0.13.1
⌘ [336ed68f] CSV v0.10.17
^ [13f3f980] CairoMakie v0.15.13
⌘ [324d7699] CategoricalArrays v0.10.9
```

```

[a93c6f00] DataFrames v1.8.2
[31c24e10] Distributions v0.25.131
[8f03c58b] Effects v1.7.1
[38e38edf] GLM v1.9.5
^ [09f84164] HypothesisTests v0.11.8
^ [ff71e718] MixedModels v5.8.2
⚡ [ce6b1742] RDatasets v0.7.7
^ [10745b16] Statistics v1.11.4
[2913bbd2] StatsBase v0.34.13
[3eaba693] StatsModels v0.7.10
[79ddc9fe] TidierCats v0.2.2
[fe2206b3] TidierData v0.17.2
[8ae5e7a9] TidierFiles v0.3.2
[337ecbd1] TidierPlots v1.0.0
[9a3f8284] Random v1.11.0
Info Packages marked with ^ and ⚡ have new versions available. Those with ^ may be upgradable
outdated`

```

This book was built with:

```
versioninfo()
```

```

Julia Version 1.12.6
Commit 15346901f00 (2026-04-09 19:20 UTC)
Build Info:
  Official https://julialang.org release
Platform Info:
  OS: Linux (x86_64-linux-gnu)
  CPU: 4 × AMD EPYC 7763 64-Core Processor
  WORD_SIZE: 64
  LLVM: libLLVM-18.1.7 (ORCJIT, znver3)
  GC: Built with stock GC
Threads: 1 default, 1 interactive, 1 GC (on 4 virtual cores)
Environment:
  JULIA_LOAD_PATH = @:@stdlib

```

3.19 Under the hood: Quarto

You do not need this section to follow the book. It is for readers who want to build the book themselves or write documents like it.

This book is written in [Quarto](#), the direct successor to [bookdown](#). Since version 1.5, Quarto executes Julia blocks natively, without going through Jupyter.

```

```${julia}
#| label: example
1 + 1
```

```

The repository's `README.md` explains how to render the book.

Challenges

1. In the REPL, press `]` to enter package mode (the prompt turns blue; Backspace leaves it). Run `activate my-test` and then add `DataFrames`: this creates an environment in a new folder, `my-test`. Inspect the `Project.toml` it generated, then run `activate .` to return to the book's environment.
2. Delete `my-test/Manifest.toml`, activate `my-test` again and run `resolve`. What changes?
3. Run `first-steps.jl` (Section [3.16](#)) cell by cell. In the Workspace panel, which types did `CSV.jl` assign to the `region` and `canopy` columns, and why is the type of `canopy` different from that of `area_m2`?

4 Introduction to Julia

This chapter introduces the language for readers who already program in R. The focus is on the differences that matter in practice, not an exhaustive sweep of the syntax.

```
using Statistics
using Random
Random.seed!(42)
```

TaskLocalRNG()

i How the exercises work

Most sections end with a *Your turn* box. Work on them in VS Code with the book folder open (Section 3.16), and load the checker once per session:

```
include("exercises/check.jl")
```

Then pass your answer — a value, or a function you wrote — with the exercise number:

```
check("4.3", simpson)
```

check says whether the answer is right and, if not, what it did and what was expected. Each box has a hint and a solution folded away: try first, then peek.

4.1 Vectors and indexing

```
heights = [12.4, 8.1, 15.7, 9.3, 11.0]

length(heights), first(heights), last(heights)
```

(5, 12.4, 11.0)

Two immediate differences from R:

```
heights[1]      # indexing starts at 1, as in R
heights[2:4]    # ranges are inclusive at both ends
```

```
3-element Vector{Float64}:  
 8.1  
15.7  
 9.3
```

```
# Julia does NOT silently recycle vectors:  
try  
    [1, 2, 3] + [1, 2]  
catch e  
    e  
end
```

DimensionMismatch("a has size (3,), b has size (2,), mismatch at dim 1")

In R

`c(1,2,3) + c(1,2)` recycles the shorter vector and emits only a warning; Julia raises `DimensionMismatch`, as the block above shows. The difference is one of policy: R prioritises carrying on, Julia prioritises failing early. Which serves you better depends on your workflow — what matters is knowing which one you are using.

Your turn 4.1 — Drop a bad value

These are the areas (m²) of five ponds. The last one is a data-entry error.

```
areas = [174.1, 161.6, 233.9, 174.5, 1980.0]
```

```
5-element Vector{Float64}:  
174.1  
161.6  
233.9  
174.5  
1980.0
```

Create `clean`, holding every pond except the last. Do not type the position 4: your code should still work if more ponds are added.

Hint

Inside square brackets, `end` means “the last position”, so `end - 1` is the one before it.

Solution

```
clean = areas[1:end-1]  
check("4.1", clean)
```

✓ Correct!

4.2 Broadcasting: the dot

In R, nearly every function is vectorized by default. In Julia you declare vectorization with a dot. This is explicit and works for *any* function, including the ones you write.

```
log.(heights)           # elementwise log
heights .^ 2             # elementwise power
heights .> 10            # elementwise comparison
```

5-element BitVector:

```
1
0
1
0
1
```

```
# works identically with your own function
standardize(x, μ, σ) = (x - μ) / σ

standardize.(heights, mean(heights), std(heights))
```

5-element Vector{Float64}:

```
0.3724002529860428
-1.0833461905048525
1.4896010119441712
-0.6770913690655326
-0.10156370535983013
```

The dot rule

If you mean “apply this to each element”, add a dot. $f.(x)$, $x .+ y$, $x .== y$. An entire chain can be dotted at once with `@.`:

```
@. log(heights) + 1
```

5-element Vector{Float64}:

```
3.517696472610991
3.0918640616783932
3.7536607123542622
3.2300144001592104
3.3978952727983707
```

Your turn 4.2 — Log-transform the areas

The area plot in Section 3.16 used a log scale, because pond areas are strongly right-skewed. Compute the base-10 logarithm of every value in areas and store it

in log_areas.

Hint

log10(areas) fails. Read the error, then reread the dot rule just above.

Solution

```
log_areas = log10.(areas)
check("4.2", log_areas)
```

✓ Correct!

log10 is defined for a single number; the dot applies it to each element.

4.3 Functions

```
# short form, for one-line functions
richness(abundances) = count(>(0), abundances)

# long form
function shannon(abundances)
    total = sum(abundances)
    p = abundances ./ total
    p = filter(>(0), p)          # log(0) is undefined
    return -sum(p .* log.(p))
end

community = [45, 12, 0, 8, 23, 0, 3]
richness(community), shannon(community)
```

(5, 1.289250407583511)

i Your turn 4.3 — Simpson's index

Write `simpson(abundances)`, returning Simpson's index $1 - \sum p_i^2$, where p_i is the proportion of individuals belonging to species i . The first lines of `shannon` already compute the proportions.

Before calling `check`, try `simpson([3, 1])` yourself: it should give 0.375.

Hint

`p .^ 2` squares each proportion and `sum` adds them up. Zeros need no special care here: $0^2 = 0$.

Solution

```
function simpson(abundances)
    p = abundances ./ sum(abundances)
    return 1 - sum(p.^2)
end

check("4.3", simpson)
```

✓ Correct!

4.4 Multiple dispatch

This is the central idea of the language, and the reason Julia packages compose without glue code. A function can have several **methods**, and Julia picks which one to run based on the types of *all* the arguments.

```
# one method for integer counts
diversity(x::Vector{<:Integer}) = "index for count data"

# another for proportional cover
diversity(x::Vector{<:AbstractFloat}) = "index for cover data"

diversity([45, 12, 8]), diversity([0.45, 0.12, 0.08])
```

("index for count data", "index for cover data")

In practice this means that when someone defines a new ecological data type, `mean`, `plot` and `fit` start working with it as soon as the appropriate methods are defined — without the authors of `Statistics` or `Makie` knowing the new type exists.

i Your turn 4.4 — One index per site

`simpson` handles one community. Field data usually come as a site $\times$ species matrix, one row per site. Add a second method, `simpson(M::Matrix)`, that returns one index per row. The vector method must keep working.

Hint

`eachrow(M)` goes through the rows one at a time. Combine it with the dot, or with a comprehension: `[f(row) for row in eachrow(M)]`. If `check` complains about the argument types, read its message carefully — it is a common trap.

Solution

```

simpson(M::Matrix) = simpson.(eachrow(M))

sites = [45 12 0 8;
         5 5 5 5;
         30 0 0 0]
simpson(sites)

```

```

3-element Vector{Float64}:
 0.47147928994082844
 0.75
 0.0

```

```

check("4.4", simpson)
length(methods(simpson))

```

✓ Correct!

2

The matrix method reuses the vector method for each row. One function, two methods, and Julia picks by the argument's type; run `methods(simpson)` in the REPL to see both.

4.5 Missing data

```

measurements = [12.4, missing, 15.7, 9.3]

mean(measurements)           # propagates, as in R

```

missing

```

mean(skipmissing(measurements)) # equivalent to na.rm = TRUE

```

12.466666666666669

In R

NA corresponds to missing. The difference is that missing is a type of its own, and the compiler knows whether a column can contain it — a `Vector{Float64}` never contains missing, a `Vector{Union{Missing,Float64}}` may. That eliminates a whole class of silent bugs.

i Your turn 4.5 — How much is missing?

Canopy cover was not recorded at every pond:

```
canopy = [70.0, 42.6, missing, 49.4, missing, 81.2]
```

6-element Vector{Union{Missing, Float64}}:

```
70.0  
42.6  
missing  
49.4  
missing  
81.2
```

Write `missing_summary(x)` returning a tuple of two numbers: how many values are missing, and the mean of the rest. For `canopy` that is (2, 60.8).

Hint

`count(f, x)` counts the elements for which `f` is true, and `ismissing` is such a function. The mean is in the block just above. Return both with `return (n, m)`.

Solution

```
missing_summary(x) = (count(ismissing, x),  
    ↪ mean(skipmissing(x)))  
  
missing_summary(canopy)
```

(2, 60.8)

```
check("4.5", missing_summary)
```

✓ Correct!

4.6 Chaining

```
# the native pipe |> chains single-argument functions  
community |> shannon |> exp    # Hill number of order 1
```

3.63006446685595

For pipelines with multiple arguments, `@chain` (from `Chain.jl`, re-exported by `TidierData.jl`) plays the role of R's `|>`. It is the backbone of Chapter 5.

4.7 Control flow

```
result = String[]

for (i, a) in enumerate(community)
  if a == 0
    push!(result, "species $i absent")
  elseif a < 10
    push!(result, "species $i rare")
  else
    push!(result, "species $i common")
  end
end

result
```

```
7-element Vector{String}:
"species 1 common"
"species 2 common"
"species 3 absent"
"species 4 rare"
"species 5 common"
"species 6 absent"
"species 7 rare"
```

Loops are fast

Unlike R, you do not need to vectorize for performance. Write the loop that expresses the idea. This changes how null models and simulations are written — a topic that returns in the community-ecology chapters of a future edition.

Your turn 4.6 — Shannon, one species at a time

Rewrite `shannon` as `shannon_loop`, using a `for` loop instead of broadcasting. It must give the same result as `shannon`, including for communities with zero counts such as `community`.

Hint

Start with $H = 0.0$, go through the counts one by one, turn each into a proportion and subtract $p * \log(p)$ from H . If you get `NaN`, type $0 * \log(0)$ in the REPL.

Solution

```

function shannon_loop(abundances)
    total = sum(abundances)
    H = 0.0
    for a in abundances
        a == 0 && continue      # absent species
        ↪ contribute nothing
        p = a / total
        H -= p * log(p)
    end
    return H
end

shannon_loop(community), shannon(community)

```

(1.289250407583511, 1.289250407583511)

```
check("4.6", shannon_loop)
```

✓ Correct!

`a == 0 && continue` is Julia shorthand for `if a == 0 continue end`.

4.8 Naming conventions

| Convention | Meaning |
|--------------------|--|
| <code>f!(x)</code> | modifies <code>x</code> in place (sort!, push!) |
| <code>f.(x)</code> | applies <code>f</code> elementwise |
| <code>@f x</code> | a macro, not a function — rewrites code before running |
| Type | types in CamelCase |
| function_name | functions lowercase, usually without separators |

Challenges

These have no checker and no single right answer. They are about what your code does — and what it does not tell you.

- `simpson([-3, 5])` returns a number. Which number, and should it? Change the vector method of `simpson` so that negative counts raise an informative error with `throw(ArgumentError("..."))`. Does the matrix method inherit the fix?
- Communities `[10, 4, 3, 3]` and `[8, 7, 4, 1]` have the same richness and the same number of individuals. Compute Shannon and Simpson for both. Do the two

indices agree on which community is more diverse? What does this imply for a paper that reports “diversity” as a single number?

3. Load `data/ponds.csv` as in Section 3.16 and run `missing_summary` on `ponds.canopy`. Does the count agree with what `describe(ponds)` reports?

Part II

Wrangling and exploring data

5 Data wrangling with Tidier

In the original book this is the tidyverse chapter. Julia has a remarkably faithful translation: the **Tidier** family, a pure-Julia reimplementaion of the tidyverse mini-language on top of DataFrames.jl.

```
using TidierData
using DataFrames
using Statistics
```

5.1 The data

Throughout the book we use a **simulated** survey of anurans in 180 ponds across three regions. The data are simulated on purpose: because the generating process is known, every model fitted in Chapter 8 and Chapter 9 can be checked against the truth. The generator lives in scripts/make_data.jl.

```
using CSV

ponds = CSV.read(joinpath("../", "data", "ponds.csv"), DataFrame)
first(ponds, 5)
```

| | pond_id | region | hydro | area_m2 | canopy | temp_c | richness | focal | body_size |
|---|---------|----------|-----------|---------|----------|---------|----------|-------|-----------|
| | String7 | String15 | String15 | Float64 | Float64? | Float64 | Int64 | Int64 | Float64 |
| 1 | P001 | Atlantic | permanent | 174.1 | 70.0 | 22.4 | 8 | 0 | 34.55 |
| 2 | P002 | Amazon | permanent | 161.6 | 42.6 | 23.9 | 12 | 1 | 30.36 |
| 3 | P003 | Cerrado | temporary | 233.9 | 57.5 | 25.2 | 6 | 1 | 34.44 |
| 4 | P004 | Amazon | temporary | 174.5 | 49.4 | 25.6 | 13 | 1 | 39.3 |
| 5 | P005 | Cerrado | permanent | 314.4 | 30.0 | 27.6 | 6 | 0 | 43.65 |

```
describe(ponds, :eltype, :nmissing, :mean, :min, :max)
```

| | variable | eltype | nmissing | mean | min | max |
|---|-----------|-------------------------|----------|----------|-----------|-----------|
| | Symbol | Type | Int64 | Union... | Any | Any |
| 1 | pond_id | String7 | 0 | | P001 | P180 |
| 2 | region | String15 | 0 | | Amazon | Cerrado |
| 3 | hydro | String15 | 0 | | permanent | temporary |
| 4 | area_m2 | Float64 | 0 | 240.215 | 10.0 | 1981.8 |
| 5 | canopy | Union{Missing, Float64} | 7 | 49.1422 | 6.4 | 95.8 |
| 6 | temp_c | Float64 | 0 | 24.4406 | 17.0 | 33.9 |
| 7 | richness | Int64 | 0 | 7.13333 | 0 | 31 |
| 8 | focal | Int64 | 0 | 0.45 | 0 | 1 |
| 9 | body_size | Float64 | 0 | 35.6777 | 26.91 | 44.26 |

| Column | Meaning |
|-----------|------------------------------------|
| pond_id | pond identifier |
| region | region (Atlantic, Cerrado, Amazon) |
| hydro | hydroperiod (permanent, temporary) |
| area_m2 | pond area (m ²) |
| canopy | canopy cover (%) |
| temp_c | water temperature (°C) |
| richness | species richness (count) |
| focal | focal species occurrence (0/1) |
| body_size | mean body size (mm) |

i Exercises in this chapter

The *Your turn* boxes work as in Chapter 4. Here the answers are tables and numbers computed from the pond data, so start each session by loading the checker, the packages and the data, with the book folder open in VS Code:

```
include("exercises/check.jl")
using TidierData, DataFrames, Statistics, CSV
ponds = CSV.read(joinpath("data", "ponds.csv"), DataFrame)
```

i Note

Column names are in English in both versions of the book, so that the code is literally identical in English and Portuguese.

5.2 @chain: the pipe

@chain is the backbone of everything that follows. It passes each line's result as the first argument of the next — exactly the role of |> (or %>%) in R.

```
@chain ponds begin
  @filter(region == "Amazon")
  @select(pond_id, area_m2, canopy, richness)
  @arrange(desc(richness))
  first(5)
end
```

| | pond_id | area_m2 | canopy | richness |
|---|---------|---------|----------|----------|
| | String7 | Float64 | Float64? | Int64 |
| 1 | P065 | 1048.6 | 18.0 | 31 |
| 2 | P044 | 568.1 | 31.7 | 24 |
| 3 | P067 | 411.4 | 30.9 | 21 |
| 4 | P085 | 204.6 | 58.2 | 19 |
| 5 | P073 | 140.5 | 17.7 | 18 |

In R

```
ponds %>%  
  filter(region == "Amazon") %>%  
  select(pond_id, area_m2, canopy, richness) %>%  
  arrange(desc(richness)) %>%  
  head(5)
```

The correspondence is line for line. The only syntactic difference is the @ and the begin ... end.

i Your turn 5.1 — The largest ponds in the Cerrado

Write a @chain that returns the three largest ponds (by area_m2) in the Cerrado, largest first. Store the result in top3 and check it with check("5.1", top3).

Hint

Three steps, one per line: keep the Cerrado rows, sort by area from largest to smallest, keep the first three. The block above does something very similar.

Solution

```
top3 = @chain ponds begin  
  @filter(region == "Cerrado")  
  @arrange(desc(area_m2))  
  first(3)  
end
```

| | pond_id | region | hydro | area_m2 | canopy | temp_c | richness | focal | body_size |
|---|---------|----------|-----------|---------|----------|---------|----------|-------|-----------|
| | String7 | String15 | String15 | Float64 | Float64? | Float64 | Int64 | Int64 | Float64 |
| 1 | P130 | Cerrado | temporary | 1981.8 | 84.9 | 26.0 | 9 | 0 | 38.53 |
| 2 | P034 | Cerrado | temporary | 1126.3 | missing | 22.9 | 6 | 0 | 33.53 |
| 3 | P143 | Cerrado | temporary | 932.7 | 39.8 | 25.1 | 11 | 0 | 34.15 |

```
check("5.1", top3)
```

✓ Correct!

5.3 The five verbs

5.3.1 @filter — pick rows

```
@chain ponds begin  
  @filter(richness > 8 && hydro == "permanent")  
  nrow  
end
```

Note: inside `@filter`, use `&&` and `||`, not `&` and `|` as in `dplyr`.

i Your turn 5.2 — Several conditions at once

How many temporary ponds have canopy cover above 60% **and** richness below the overall median richness? Check the number with `check("5.2", n)`.

Hint

Compute the median first, outside the chain: `med = median(ponds.richness)`. Inside a Tidier macro a bare name means a column, so to use the variable `med` write `!!med`.

If you get `TypeError: non-boolean (Missing) used in boolean context`, some ponds have no canopy value, and `missing > 60` is neither true nor false. Remove those rows first with `@drop_missing(canopy)` (Section 5.4).

Solution

```
med = median(ponds.richness)

n = @chain ponds begin
  @drop_missing(canopy)
  @filter(hydro == "temporary" && canopy > 60 &&
    ↪ richness < !!med)
  nrow
end
```

18

```
check("5.2", n)
```

✓ Correct!

Dropping ponds without canopy is the right call here: we cannot know whether their canopy is above 60%. The median, though, is computed on all 180 ponds, before dropping anything.

5.3.2 `@select` — pick columns

```
@chain ponds begin
  @select(pond_id, region, richness)
  first(3)
end
```

| | pond_id | region | richness |
|---|---------|----------|----------|
| | String7 | String15 | Int64 |
| 1 | P001 | Atlantic | 8 |
| 2 | P002 | Amazon | 12 |
| 3 | P003 | Cerrado | 6 |

```
# exclusion and ranges work as in dplyr
@chain ponds begin
  @select(-pond_id, -focal)
  names
end
```

9-element Vector{String}:

```
"region"
"hydro"
"area_m2"
"canopy"
"temp_c"
"richness"
"focal"
"body_size"
"pond_id"
```

5.3.3 @mutate — create columns

```
ponds_t = @chain ponds begin
  @mutate(
    log_area = log(area_m2),
    canopy_std = (canopy - mean(skipmissing(canopy))) /
    ↪ std(skipmissing(canopy)),
    open = canopy < 40,
  )
end

@chain ponds_t begin
  @select(pond_id, area_m2, log_area, canopy, canopy_std, open)
  first(5)
end
```

| | pond_id | area_m2 | log_area | canopy | canopy_std | open |
|---|---------|---------|----------|----------|------------|-------|
| | String7 | Float64 | Float64 | Float64? | Float64? | Bool? |
| 1 | P001 | 174.1 | 5.15963 | 70.0 | 0.946503 | 0 |
| 2 | P002 | 161.6 | 5.08512 | 42.6 | -0.296877 | 0 |
| 3 | P003 | 233.9 | 5.45489 | 57.5 | 0.379268 | 0 |
| 4 | P004 | 174.5 | 5.16192 | 49.4 | 0.0116988 | 0 |
| 5 | P005 | 314.4 | 5.75067 | 30.0 | -0.868651 | 1 |

i Your turn 5.3 — Richness per 100 m²

Bigger ponds tend to hold more species. Add a column `rich_per_100m2` with the number of species per 100 m² of pond, keeping every row and column of ponds. Store the result in `ponds_d` and check it with `check("5.3", ponds_d)`.

Hint

`richness / area_m2` is species per m²; multiply by 100.

Solution

```
ponds_d = @chain ponds begin
  @mutate(rich_per_100m2 = richness / area_m2 * 100)
end

check("5.3", ponds_d)
```

✓ Correct!

5.3.4 @group_by + @summarize — aggregate

```
@chain ponds begin
  @group_by(region, hydro)
  @summarize(
    n          = n(),
    mean_rich  = mean(richness),
    sd_rich    = std(richness),
    prop_focal = mean(focal),
  )
  @arrange(region, hydro)
end
```

GroupedDataFrame with 3 groups based on key: region

First Group (2 rows): region = "Amazon"

| | region | hydro | n | mean_rich | sd_rich | prop_focal |
|-----|----------|-----------|-------|-----------|---------|------------|
| | String15 | String15 | Int64 | Float64 | Float64 | Float64 |
| 1 | Amazon | permanent | 26 | 11.2692 | 5.99038 | 0.538462 |
| 2 | Amazon | temporary | 22 | 10.9091 | 5.95946 | 0.318182 |
| ... | | | | | | |

Last Group (2 rows): region = "Cerrado"

| | region | hydro | n | mean_rich | sd_rich | prop_focal |
|---|----------|-----------|-------|-----------|---------|------------|
| | String15 | String15 | Int64 | Float64 | Float64 | Float64 |
| 1 | Cerrado | permanent | 31 | 4.77419 | 2.90643 | 0.580645 |
| 2 | Cerrado | temporary | 35 | 4.8 | 2.95854 | 0.4 |

i Your turn 5.4 — Where is the focal species?

Compute, for each region, the proportion of ponds where the focal species occurs, in a column called `prop_focal`, sorted from highest to lowest. Check it with `check("5.4", by_region)`.

Hint

`focal` is 0 or 1, so its mean is the proportion of ponds where it is 1. Sorting from highest to lowest is `@arrange(desc(...))`.

Solution

```
by_region = @chain ponds begin
  @group_by(region)
  @summarize(prop_focal = mean(focal))
  @arrange(desc(prop_focal))
end

check("5.4", by_region)
```

✓ Correct!

5.3.5 @arrange — sort

```
@chain ponds begin
  @arrange(region, desc(richness))
  @select(region, pond_id, richness)
  first(6)
end
```

| | region | pond_id | richness |
|---|----------|---------|----------|
| | String15 | String7 | Int64 |
| 1 | Amazon | P065 | 31 |
| 2 | Amazon | P044 | 24 |
| 3 | Amazon | P067 | 21 |
| 4 | Amazon | P085 | 19 |
| 5 | Amazon | P073 | 18 |
| 6 | Amazon | P164 | 18 |

5.4 Missing data

The canopy column has missing values on purpose.

```
sum(ismissing.(ponds.canopy))
```

```
@chain ponds begin
  @drop_missing(canopy)
  nrow
end
```

173

```
# aggregating while ignoring missings
@chain ponds begin
  @group_by(region)
  @summarize(mean_canopy = mean(skipmissing(canopy)))
end
```

| | region | mean_canopy |
|---|----------|-------------|
| | String15 | Float64 |
| 1 | Atlantic | 51.7125 |
| 2 | Amazon | 45.313 |
| 3 | Cerrado | 49.327 |

⚠ Warning

`mean(canopy)` returns missing if any value is missing — the same behaviour as R's `mean()` without `na.rm = TRUE`. Be explicit with `skipmissing`.

i Your turn 5.5 — Where are the gaps?

Before dropping missing values, it is worth knowing where they are: if they concentrate in one region, dropping them changes that region more than the others. Count, per region, how many ponds have no canopy value, in a column called `n_missing`. Check it with `check("5.5", gaps)`.

Hint

`ismissing.(canopy)` gives true for each missing value, and `sum` counts the trues. Both work inside `@summarize`.

Solution

```
gaps = @chain ponds begin
  @group_by(region)
  @summarize(n_missing = sum(ismissing.(canopy)))
end

check("5.5", gaps)
```

✓ Correct!

Seven missing values spread over three regions: no region is hit much harder than the others, so dropping them is unlikely to bias comparisons between regions.

5.5 Long and wide formats

```
long = @chain ponds begin
  @select(pond_id, region, area_m2, canopy, temp_c)
  @pivot_longer(area_m2:temp_c, names_to = "variable", values_to =
    ↪ "value")
end

first(long, 6)
```

| | pond_id | region | variable | value |
|---|---------|----------|----------|----------|
| | String7 | String15 | String | Float64? |
| 1 | P001 | Atlantic | area_m2 | 174.1 |
| 2 | P002 | Amazon | area_m2 | 161.6 |
| 3 | P003 | Cerrado | area_m2 | 233.9 |
| 4 | P004 | Amazon | area_m2 | 174.5 |
| 5 | P005 | Cerrado | area_m2 | 314.4 |
| 6 | P006 | Cerrado | area_m2 | 620.2 |

```
@chain long begin
  @pivot_wider(names_from = variable, values_from = value)
  first(3)
end
```

| | pond_id | region | area_m2 | canopy | temp_c |
|---|---------|----------|----------|----------|----------|
| | String7 | String15 | Float64? | Float64? | Float64? |
| 1 | P001 | Atlantic | 174.1 | 70.0 | 22.4 |
| 2 | P002 | Amazon | 161.6 | 42.6 | 23.9 |
| 3 | P003 | Cerrado | 233.9 | 57.5 | 25.2 |

5.6 Joins

```
# an auxiliary table with region-level information
regions = DataFrame(
  region      = ["Atlantic", "Cerrado", "Amazon"],
  biome_type  = ["forest", "savanna", "forest"],
  mean_rain   = [1400, 1500, 2300],
)

@chain ponds begin
```

```

@left_join(regions, region)
@select(pond_id, region, biome_type, mean_rain, richness)
first(5)
end

```

| | pond_id | region | biome_type | mean_rain | richness |
|---|---------|----------|------------|-----------|----------|
| | String7 | String15 | String? | Int64? | Int64 |
| 1 | P001 | Atlantic | forest | 1400 | 8 |
| 2 | P002 | Amazon | forest | 2300 | 12 |
| 3 | P003 | Cerrado | savanna | 1500 | 6 |
| 4 | P004 | Amazon | forest | 2300 | 13 |
| 5 | P005 | Cerrado | savanna | 1500 | 6 |

i Your turn 5.6 — Forest versus savanna

Using the regions table above, compute the mean richness of forest and savanna ponds, in a column called mean_rich. Check it with check("5.6", by_biome).

Hint

Join first, so every pond gets its biome_type; then group by biome_type and summarize.

Solution

```

by_biome = @chain ponds begin
  @left_join(regions, region)
  @group_by(biome_type)
  @summarize(mean_rich = mean(richness))
end

check("5.6", by_biome)

```

✓ Correct!

5.7 DataFrames.jl underneath

Tidier is a convenience layer. It is worth knowing the native form, which is what you will meet in most of the ecosystem's documentation:

```

combine(
  groupby(ponds, :region),
  :richness => mean => :mean_rich,
  :richness => length => :n,
)

```

| | region | mean_rich | n |
|---|----------|-----------|-------|
| | String15 | Float64 | Int64 |
| 1 | Atlantic | 6.59091 | 66 |
| 2 | Amazon | 11.1042 | 48 |
| 3 | Cerrado | 4.78788 | 66 |

Same result as the `@group_by/@summarize` above. Use Tidier to write, and recognize the native form to read.

i Your turn 5.7 — The same, natively

Rewrite your answer to 5.4 with `groupby`, `combine` and `sort` from `DataFrames.jl`, without Tidier macros. Check it with `check("5.7", by_region_native)`.

Hint

`sort(df, :col, rev = true)` sorts from highest to lowest.

Solution

```
by_region_native = sort(
  combine(groupby(ponds, :region), :focal => mean =>
    ↪ :prop_focal),
  :prop_focal, rev = true)

check("5.7", by_region_native)
```

✓ Correct!

Challenges

These have no checker. They are about what the numbers do — and do not — tell you.

1. In 5.4 the Cerrado comes first, but the three proportions only range from 0.42 to 0.48, each from 48 to 66 ponds. Open `scripts/make_data.jl`: does region affect the focal species in the simulation? What does that tell you about reading a ranking off a summary table?
2. In 5.6 forest ponds are richer than savanna ponds. But “forest” here means Atlantic plus Amazon. Compute mean richness per region and decide whether “forest” is a useful grouping for this question.
3. Use `@pivot_longer` to put `area_m2`, `canopy` and `temp_c` in one column, then compute the mean of each variable per region in a single pipeline. How do you deal with the missing canopy values?

6 Data visualization

Julia has three reasonable routes to graphics, all resting on [Makie.jl](#) (Danisch and Krumbiegel 2021):

| Route | For whom |
|----------------------|---|
| TidierPlots.jl | people coming from ggplot2 who want the same grammar |
| AlgebraOfGraphics.jl | people who want an idiomatically Julian grammar of graphics |
| Plain Makie.jl | full control over the figure |

This chapter shows all three on the same data, because the right choice depends on the task — and because knowing how to translate between them keeps you from getting stuck.

```
using TidierData
using CairoMakie
using AlgebraOfGraphics
using CSV, DataFrames

CairoMakie.activate!(type = "png")

ponds = CSV.read(joinpath("../", "data", "ponds.csv"), DataFrame)
ponds_c = @chain ponds begin
    @drop_missing(canopy)
    @mutate(log_area = log(area_m2))
end

nrow(ponds_c)
```

173

i Exercises in this chapter

A figure cannot be checked automatically, so in most *Your turn* boxes you compare your plot with the one in the solution. Start each session by loading the packages and the data, with the book folder open in VS Code:

```
include("exercises/check.jl")
using TidierData, TidierPlots, CairoMakie, AlgebraOfGraphics, CSV,
    DataFrames, Statistics
ponds = CSV.read(joinpath("data", "ponds.csv"), DataFrame)
ponds_c = @chain ponds begin
    @drop_missing(canopy)
    @mutate(log_area = log(area_m2))
end
```

6.1 TidierPlots.jl: the ggplot grammar

```
using TidierPlots

ggplot(ponds_c, @aes(x = canopy, y = richness, color = region)) +
    geom_point(alpha = 0.7) +
    labs(x = "Canopy cover (%)", y = "Species richness",
         title = "Richness declines with shading")
```

Figure 6.1: Richness against canopy cover, by region.

In R

```
ggplot(ponds_c, aes(x = canopy, y = richness, colour = region)) +
    geom_point(alpha = 0.7) +
    labs(x = "Canopy cover (%)", y = "Species richness")
```

The translation is nearly character for character; the difference is the @ in @aes.

i Your turn 6.1 — Richness and temperature

Redo Figure 6.1 with water temperature (temp_c) on the x axis and the points coloured by hydroperiod (hydro). Label both axes. Does richness change with temperature as clearly as with canopy?

There is no checker for this one: compare your figure with the one in the solution.

Hint

Only the @aes mapping and the labels change.

Solution

```
ggplot(ponds_c, @aes(x = temp_c, y = richness, color =
  ↪ hydro)) +
  geom_point(alpha = 0.7) +
  labs(x = "Water temperature (°C)", y = "Species
    ↪ richness")
```

The cloud is much flatter than for canopy. The simulation does include a temperature effect (scripts/make_data.jl), but a small one: about 3% more species per degree, invisible next to the scatter between ponds.

6.2 AlgebraOfGraphics.jl: an algebraic grammar

AoG treats plots as an algebra: * combines layers, + superimposes them.

```
plt = AlgebraOfGraphics.data(ponds_c) *
  mapping(:canopy => "Canopy cover (%)",
    :richness => "Species richness",
    color = :region => "Region") *
  (visual(Scatter, alpha = 0.6) + AlgebraOfGraphics.linear())

draw(plt; figure = (size = (700, 450),))
```

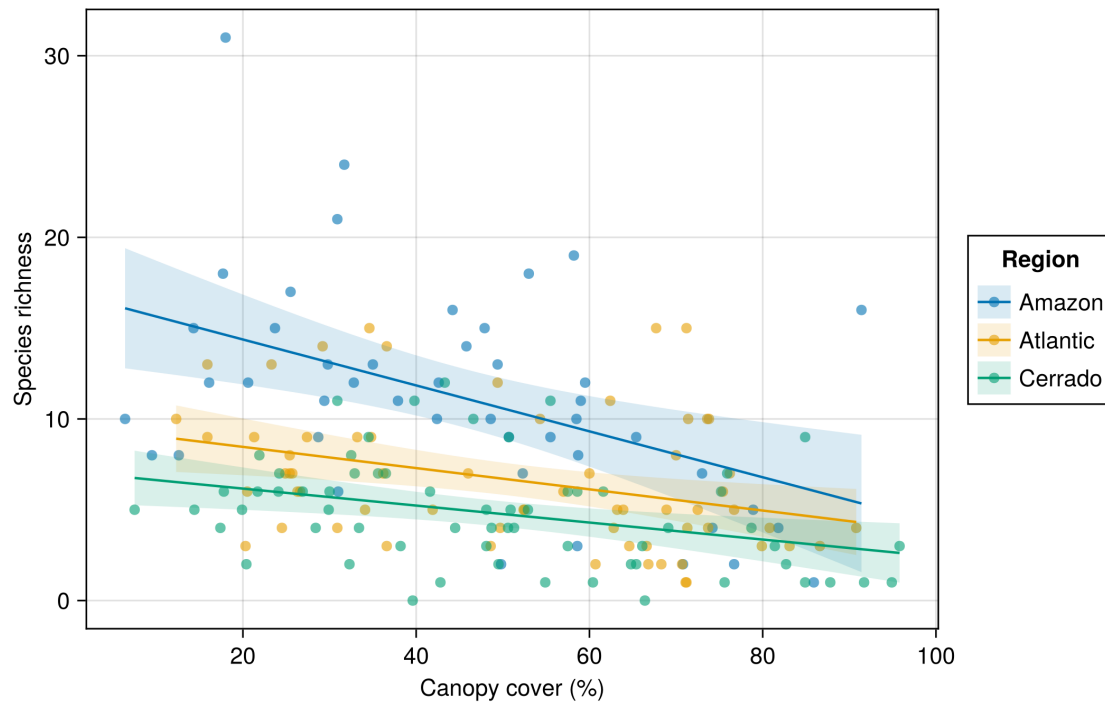


Figure 6.2: The same plot in AlgebraOfGraphics, with a linear fit per region.

i Note

`AlgebraOfGraphics.data` and `AlgebraOfGraphics.linear` are qualified because the names `data` and `linear` collide with other loaded packages. In a script using only AoG, `data(...)` and `linear()` suffice.

i Your turn 6.2 — Body size, with fitted lines

With `AlgebraOfGraphics`, plot `body_size` against `temp_c`, coloured by `hydro`, with a linear fit for each hydroperiod. Do permanent and temporary ponds differ? There is no checker for this one: compare your figure with the one in the solution.

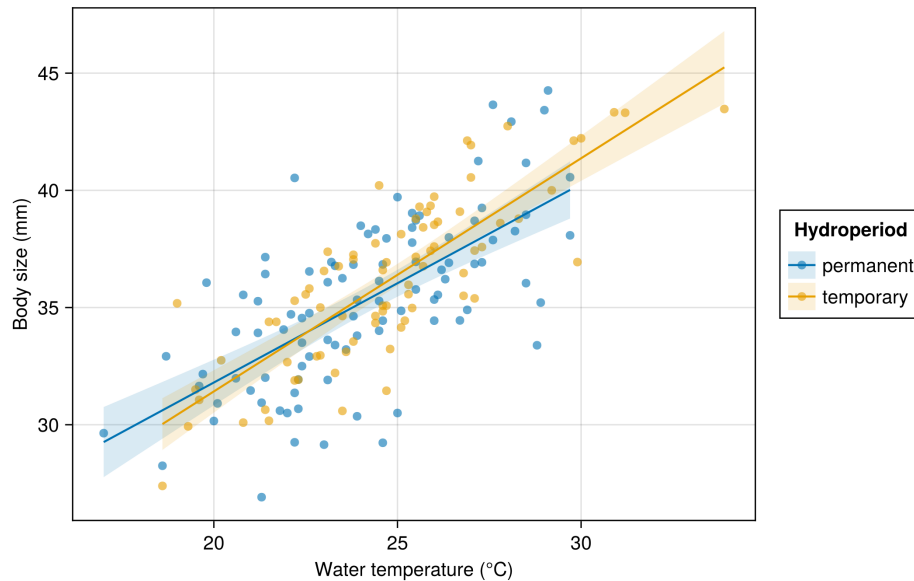
Hint

Start from the block above and change the mapping. The fitted lines follow the colour grouping automatically.

Solution

```
plt = AlgebraOfGraphics.data(ponds_c) *
  mapping(:temp_c => "Water temperature (°C)",
    :body_size => "Body size (mm)",
    color = :hydro => "Hydroperiod") *
  (visual(Scatter, alpha = 0.6) +
    ↪ AlgebraOfGraphics.linear())

draw(plt; figure = (size = (700, 450),))
```



The two lines almost coincide: in the simulation, body size depends on temperature and canopy but not on hydroperiod. We test this formally in Chapter 8.

6.3 Facets

```
plt = AlgebraOfGraphics.data(ponds_c) *
  mapping(:log_area => "log(area)",
    :richness => "Richness",
    color = :region,
    layout = :hydro) *
  (visual(Scatter, alpha = 0.6) + AlgebraOfGraphics.linear())

draw(plt; figure = (size = (800, 400),))
```

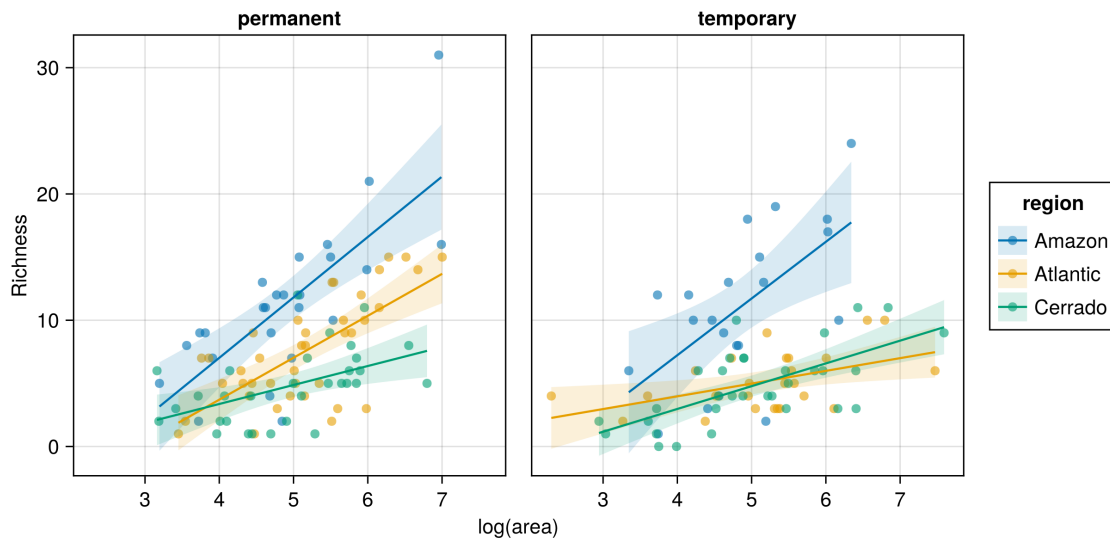


Figure 6.3: Richness-area relationship faceted by hydroperiod.

i Your turn 6.3 — Facets in the ggplot grammar

Translate Figure 6.3 to TidierPlots.jl, using `facet_wrap` to split the panels by hydroperiod. Leave out the fitted lines (see the warning in the solution). There is no checker for this one: compare your figure with the one in the solution.

Hint

`facet_wrap(facets = :hydro)` is added with `+`, like any other layer.

Solution

```
ggplot(ponds_c, @aes(x = log_area, y = richness, color =
  ↪ region)) +
  geom_point(alpha = 0.6) +
  facet_wrap(facets = :hydro) +
  labs(x = "log(area)", y = "Richness")
```

A limit of TidierPlots

In TidierPlots.jl 1.0.0 (checked in September 2026), adding `geom_smooth(method = "lm")` to a faceted plot fails with column name `:facet` not found. `geom_smooth` works without facets, and facets work without `geom_smooth`, but not together. When the ggplot grammar reaches a limit like this, AlgebraOfGraphics (Figure 6.3) is the way out.

6.4 Distributions

```
plt = AlgebraOfGraphics.data(ponds_c) *  
  mapping(:region => "Region",  
    :richness => "Richness",  
    color = :hydro => "Hydroperiod",  
    dodge = :hydro) *  
  visual(BoxPlot)  
  
draw(plt; figure = (size = (700, 420),))
```

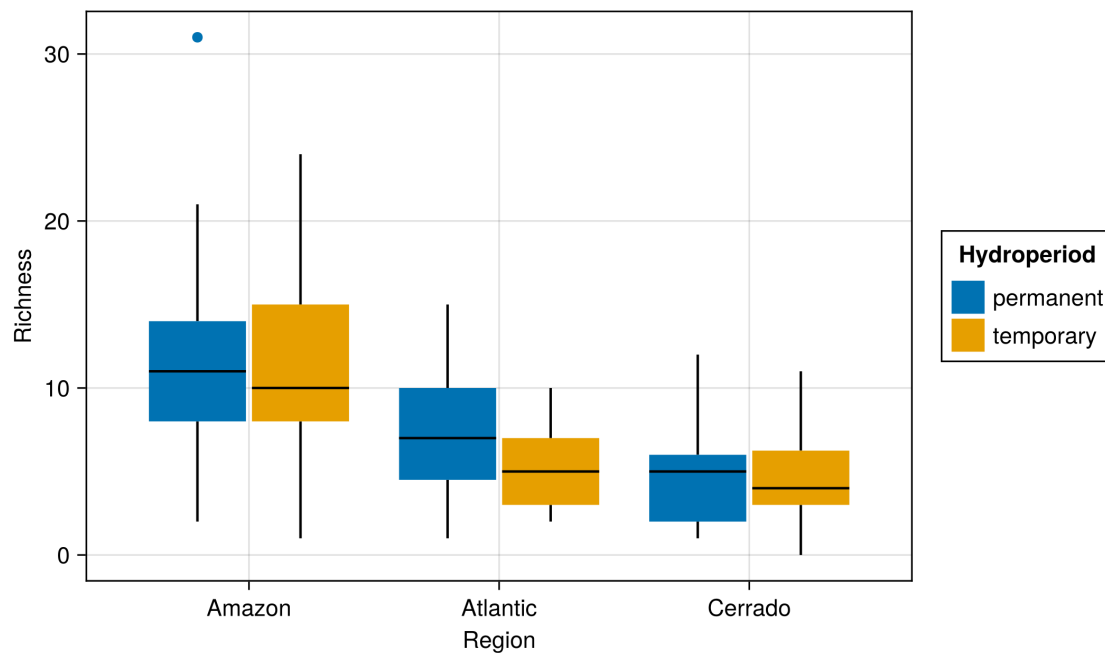


Figure 6.4: Richness distribution by region and hydroperiod.

i Your turn 6.4 — The shape of a distribution

Boxplots hide the shape of a distribution. Show the distribution of `body_size` in each region as violins (Violin), with one colour per region. There is no checker for this one: compare your figure with the one in the solution.

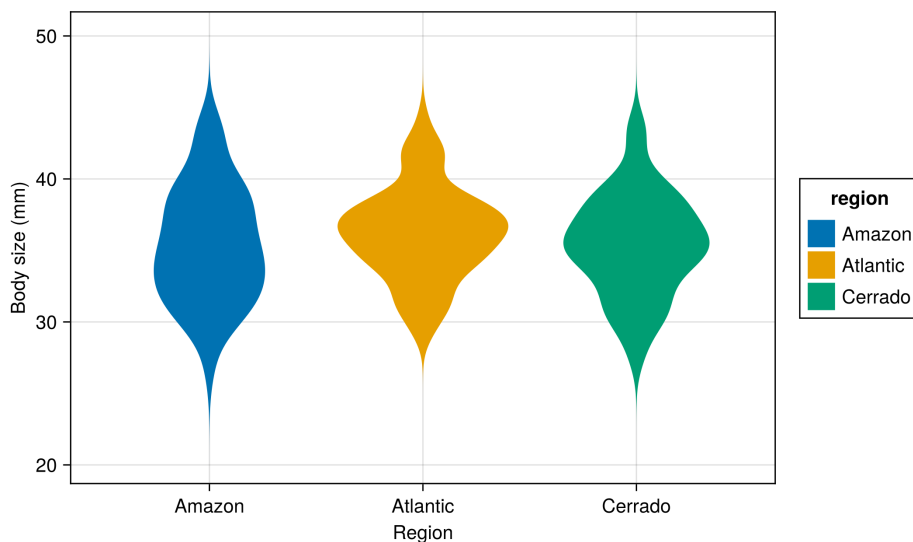
Hint

Replace `BoxPlot` with `Violin` and map `region` to both the x axis and color.

Solution

```
plt = AlgebraOfGraphics.data(ponds_c) *
  mapping(:region => "Region",
    :body_size => "Body size (mm)",
    color = :region) *
  visual(Violin)

draw(plt; figure = (size = (700, 420),))
```



6.5 Plain Makie.jl: full control

When a figure needs a layout the grammar cannot express, drop down to Makie.

```
fig = Figure(size = (860, 380))

ax1 = Axis(fig[1, 1], xlabel = "Canopy cover (%)", ylabel = "Richness",
  title = "(a) Canopy")
scatter!(ax1, ponds_c.canopy, ponds_c.richness, color = (:steelblue,
  ↪ 0.6))

ax2 = Axis(fig[1, 2], xlabel = "log(area)", ylabel = "Richness",
  title = "(b) Area")
scatter!(ax2, ponds_c.log_area, ponds_c.richness, color = (:darkorange,
  ↪ 0.6))

ax3 = Axis(fig[1, 3], xlabel = "Richness", ylabel = "Frequency",
  title = "(c) Distribution")
hist!(ax3, ponds_c.richness, bins = 12, color = (:seagreen, 0.7))
```

fig

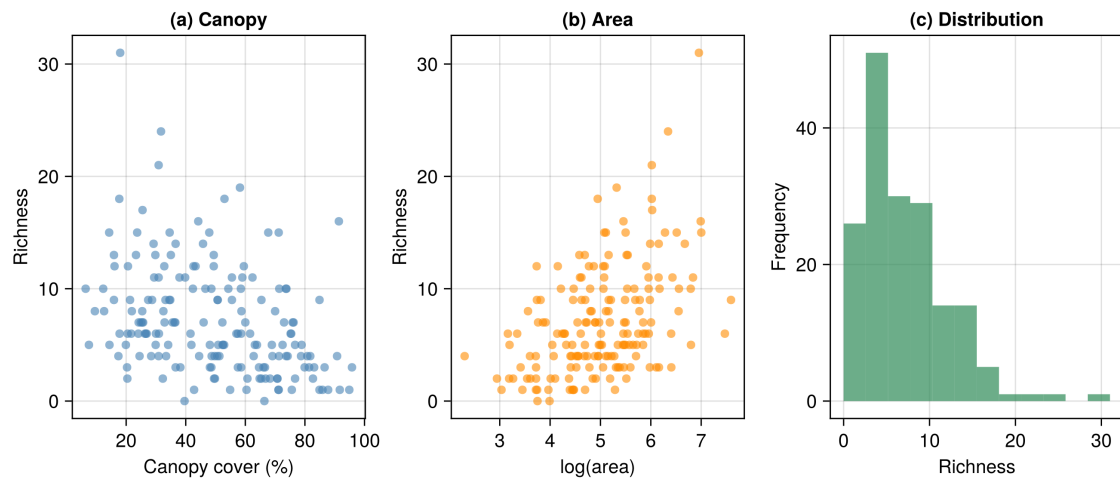


Figure 6.5: A composite figure assembled by hand with Makie.

Makie's layout system is grid-based (`fig[row, col]`), covering the role of `patchwork` and `cowplot` in R without an extra package.

i Your turn 6.5 — Two panels by hand

Build a figure with `size = (800, 350)` and two panels: on the left, a boxplot of richness by region; on the right, a bar chart of the proportion of ponds with the focal species in each region (the numbers from 5.4). Store it in `fig_regions`. There is no checker for this one: compare your figure with the one in the solution.

Hint

Makie's `boxplot!` and `barplot!` want numbers on the x axis. Give each region a number with `[findfirst(==(r), regs) for r in ponds.region]`, where `regs = ["Amazon", "Atlantic", "Cerrado"]`, and put the names back with `xticks = (1:3, regs)` in each Axis.

Solution

```

regs = ["Amazon", "Atlantic", "Cerrado"]
idx = [findfirst(==(r), regs) for r in ponds.region]
props = [mean(ponds.focal[ponds.region .== r]) for r in
  ↪ regs]

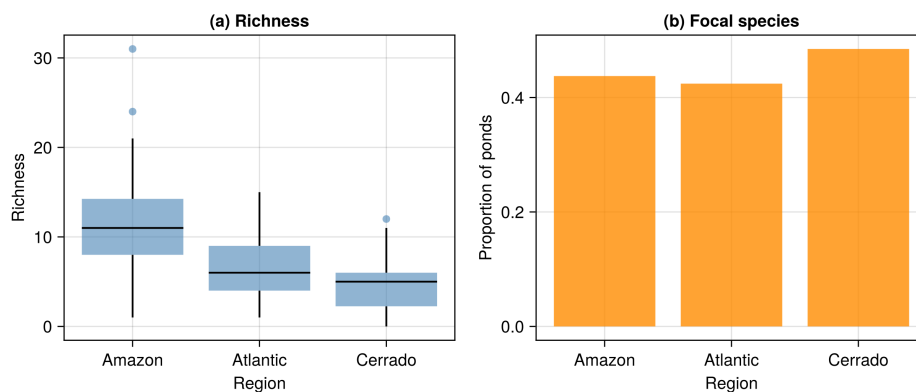
fig_regions = Figure(size = (800, 350))

ax1 = Axis(fig_regions[1, 1], xticks = (1:3, regs),
  xlabel = "Region", ylabel = "Richness", title
  ↪ = "(a) Richness")
boxplot!(ax1, idx, ponds.richness, color = (:steelblue,
  ↪ 0.6))

ax2 = Axis(fig_regions[1, 2], xticks = (1:3, regs),
  xlabel = "Region", ylabel = "Proportion of
  ↪ ponds", title = "(b) Focal species")
barplot!(ax2, 1:3, props, color = (:darkorange, 0.8))

fig_regions

```



The two panels tell different stories: richness differs sharply between regions, focal-species occurrence hardly at all.

6.6 Saving figures

```

save("figure.png", fig; px_per_unit = 3) # high-resolution raster
save("figure.pdf", fig)                  # vector, for publication
save("figure.svg", fig)

```

i Your turn 6.6 — A figure for publication

Save `fig_regions` from 6.5 as `focal-by-region.png`, at three pixels per unit, in the book folder. Then check the file: `check("6.6", "focal-by-region.png")`.

Hint

The block above shows the keyword that sets the resolution.

Solution

```
save("focal-by-region.png", fig_regions; px_per_unit = 3)
check("6.6", "focal-by-region.png")
```

An 800×350 figure at three pixels per unit gives a 2400×1050 px image — enough for a two-column journal figure. For vector output, save as `.pdf` instead.

💡 Choosing a backend

CairoMakie produces publication-quality static output (PNG, PDF, SVG) and is what this book uses. GLMakie opens GPU-accelerated interactive windows — useful for exploring data, unsuitable for rendering a book.

Challenges

1. In 6.1 you judged the temperature effect by eye. Add a linear fit per hydroperiod with `geom_smooth(method = "lm")`. Does the line change your reading of the plot? What would you need to know before trusting it?
2. Panel (b) of 6.5 shows proportions without uncertainty. Add an error bar of ± 2 standard errors to each bar, with $SE = \sqrt{p(1-p)/n}$ and `errorbars!`. Do the regions still look different?
3. Redo Figure 6.5 with one colour per region in every panel and a single shared legend on the right (`Legend(fig[1, 4], ...)`).

7 Exploratory data analysis

Before any model, look at the data. Exploration has four jobs (Zuur et al. 2010): find **errors and outliers**, understand the **structure** of the data — what counts as a replicate —, see the **distributions** that models will have to describe, and detect **collinearity** between variables that a model will struggle to separate. Skipping it does not save time: the problems are still there, and they surface later as strange model results.

The simulated ponds of the previous chapters are too clean for this. This chapter uses real data, with all the problems real data have.

i Where this chapter comes from

Like Chapter 10 and Chapter 11, this chapter is **inspired by ecoR** (Oliveira 2026), the R course wiki of the Instituto de Biociências, Universidade de São Paulo, produced and maintained by Alexandre Adalardo de Oliveira. Its sequence — checking the data frame, descriptive statistics, univariate graphs, relationships between variables, and the Anscombe quartet — follows ecoR’s exploratory-analysis module. The text, data, examples and exercises here are written from scratch for Julia; the authors of ecoR have not reviewed this book.

If you read Portuguese and want to go further, the original material is freely available (CC BY-SA 4.0):

- [4. Análise Exploratória de Dados](#) — the course notes;
- [4. Tutoriais de Análise Exploratória de Dados](#) — the step-by-step tutorial;
- [exercises on exploratory analysis](#), automatically graded in [notaR](#), in R.

i The data: ATLANTIC BIRD TRAITS

ATLANTIC BIRD TRAITS (Rodrigues et al. 2019) compiles morphological measurements of birds from the Atlantic forests of South America: 72 483 records, gathered from museum collections, ringing projects and the literature between 1820 and 2018, by 179 authors. Its data paper states that there are no copyright or proprietary restrictions on its use, and asks that it be cited when used in publications or teaching — as here.

`data/atlantic_bird_traits.csv` keeps **all** records and 21 of the original 83 columns, with their original names and codes. Nothing is cleaned: finding the problems is the point. `scripts/prepare_birds.jl` documents how the subset was made from the original file, which is in the paper’s supporting information.

```
using TidierData
using DataFrames, CSV
using Statistics, StatsBase
using GLM, RDatasets, Distributions
```

```
using CairoMakie

CairoMakie.activate!(type = "png")
bird_file = joinpath("../", "data", "atlantic_bird_traits.csv")
```

"../data/atlantic_bird_traits.csv"

i Exercises in this chapter

The answers are numbers computed from the bird data. Start each session with:

```
include("exercises/check.jl")
using TidierData, DataFrames, CSV, Statistics, StatsBase, GLM,
    ⇨ Distributions
bird_file = joinpath("data", "atlantic_bird_traits.csv")
```

The first section shows how to read the file; the later exercises assume you have read it that way, into `birds`, with the columns renamed.

7.1 First contact: reading the file

Read the file as it comes and look at the type of each column:

```
raw = CSV.read(bird_file, DataFrame)
size(raw)
```

(72483, 21)

```
first(describe(raw, :eltype, :nmissing), 8)
```

| | variable | eltype | nmissing |
|---|-------------------|----------|----------|
| | Symbol | DataType | Int64 |
| 1 | ID_ABT | String7 | 0 |
| 2 | Order | String31 | 0 |
| 3 | Family | String31 | 0 |
| 4 | Binomial | String31 | 0 |
| 5 | Body_mass.g. | String7 | 0 |
| 6 | Wing_length.mm. | String7 | 0 |
| 7 | Tail_length.mm. | String7 | 0 |
| 8 | Tarsus_length.mm. | String7 | 0 |

Body mass, wing length, tail length: all read as **text** (String), and not one value is missing. The reason is in the file itself — missing values are written as the two letters NA, which `CSV.jl` does not treat as missing by default. One non-number in a column makes the whole column text. Tell `CSV.read` which code means missing:

```
birds = CSV.read(bird_file, DataFrame; missingstring = "NA")
first(describe(birds, :eltype, :nmissing), 8)
```

| | variable | eltype | nmissing |
|---|-------------------|-------------------------|----------|
| | Symbol | Type | Int64 |
| 1 | ID_ABT | String7 | 0 |
| 2 | Order | String31 | 0 |
| 3 | Family | String31 | 0 |
| 4 | Binomial | String31 | 0 |
| 5 | Body_mass.g. | Union{Missing, Float64} | 11664 |
| 6 | Wing_length.mm. | Union{Missing, Float64} | 51134 |
| 7 | Tail_length.mm. | Union{Missing, Float64} | 26183 |
| 8 | Tarsus_length.mm. | Union{Missing, Float64} | 50742 |

In R

`read.csv` treats "NA" as missing by default, so in R this problem would pass unnoticed. Here it is visible: a numeric column that arrives as text is the first sign that the file uses a missing-value code you have not declared. Other common codes are "", "-", ".-", -999 and 99999; `missingstring` also accepts a vector of them.

The column names mix units, dots and capitals, like `Body_mass.g.`. They are awkward to type and Tidier macros cannot use them directly. Rename them once, at the start:

```
rename!(birds,
  "ID_ABT" => :id, "Order" => :order, "Family" => :family, "Binomial"
  ↪ => :species,
  "Body_mass.g." => :mass_g, "Wing_length.mm." => :wing_mm,
  "Tail_length.mm." => :tail_mm, "Tarsus_length.mm." => :tarsus_mm,
  "Bill_length.mm." => :bill_mm, "Age" => :age, "Sex" => :sex,
  ↪ "Status" => :status,
  "Recapture" => :recapture, "Ring" => :ring,
  "Latitude_decimal_degrees" => :lat, "Longitude_decimal_degrees" =>
  ↪ :lon,
  "Altitude" => :altitude, "Year" => :year, "Main_researcher" =>
  ↪ :source,
  "Outside.range" => :outside_range, "Obs.spp" => :species_note)

names(birds)
```

21-element Vector{String}:

```
"id"
"order"
"family"
"species"
"mass_g"
"wing_mm"
"tail_mm"
"tarsus_mm"
"bill_mm"
"age"
```

```
"recapture"  
"ring"  
"lat"  
"lon"  
"altitude"  
"year"  
"source"  
"outside_range"  
"species_note"
```

i Your turn 7.1 — How many birds were weighed?

How many records have a body mass? Pass the number to `check("7.1", ...)`.

Hint

`count(!ismissing, v)` counts the non-missing values of a vector. It only means something once the file has been read with `missingstring = "NA"`.

Solution

```
n_mass = count(!ismissing, birds.mass_g)
```

60819

```
check("7.1", n_mass)
```

✓ Correct!

60 819 of 72 483: body mass is the most commonly recorded trait, and still missing for one record in six.

7.2 Missing, unknown, and inconsistent

Categorical columns deserve a table of their levels before anything else:

```
(age = countmap(birds.age), sex = countmap(birds.sex))
```

```
(age = Dict{Union{Missing, String15}, Int64}{String15("Juvenile") => 8708, missing => 4935
```

There are **two kinds of missing** here. `missing` means the information was never recorded; `"Unknown"` means someone looked at the bird and could not tell. They are

not the same thing: a study of sex ratios should count the unknowns, but not the unrecorded.

Other columns disagree with themselves about spelling:

```
(recapture = countmap(birds.recapture), outside_range =  
  ↪ countmap(birds.outside_range))
```

```
(recapture = Dict{Union{Missing, String3}, Int64}(String3("Yes") => 8556, missing => 33682
```

"no" and "Yes": the capitalization differs between levels. Here it is harmless, because each level has only one spelling, but a single "yes" somewhere would create a third level that grouping would treat as different. Normalizing is cheap:

```
birds.recapture      = passmissing(lowercase).(birds.recapture)  
birds.outside_range = lowercase.(birds.outside_range)  
unique(birds.recapture)
```

```
3-element Vector{Union{Missing, String}}:  
 "no"  
 missing  
 "yes"
```

`passmissing(f)` applies `f` to values and leaves missing alone.

i Your turn 7.2 — How often can sex be told?

Among records where sex was **recorded** (not missing), what proportion were sexed as "Male" or "Female"? Pass it to `check("7.2", ...)`.

Hint

Keep the non-missing values with `collect(skipmissing(birds.sex))`, then take the mean of `in(("Male", "Female")).(...)`.

Solution

```
sex_recorded = collect(skipmissing(birds.sex))  
p_sexed = mean(in(("Male", "Female")).(sex_recorded))
```

```
0.47551540822059907
```

```
check("7.2", p_sexed)
```

✓ Correct!

Less than half: many Atlantic forest passerines cannot be sexed in the hand. Any analysis by sex is an analysis of a non-random half of the data.

7.3 Structure: what is a replicate?

A row is a record, but records are not independent observations. They are grouped in several ways at once.

By species. The records are very unequally spread:

```
per_species = sort(combine(groupby(birds, :species), nrow => :n), :n,
  ↪ rev = true)

(names = nrow(per_species),
 top_five = first(per_species, 5),
 with_one_record = count(==1), per_species.n))
```

(names = 780, top_five = 5×2 DataFrame

| Row | species
String31 | n
Int64 |
|-----|--------------------------|-----------------------------|
| 1 | Chiroxiphia caudata | 1857 |
| 2 | Turdus albicollis | 1686 |
| 3 | Trichothraupis melanops | 1493 |
| 4 | Turdus leucomelas | 1475 |
| 5 | Basileuterus culicivorus | 1419, with_one_record = 68) |

780 names — some identified only to genus, such as *Elaenia* sp. — while the data paper reports 711 species. Before counting species, decide what counts as one. The ten most recorded species hold a fifth of all records; 68 names appear once.

By individual. A ringed bird caught again appears again:

```
ringed = dropmissing(birds[:, [:ring, :species]], :ring)
per_ring = combine(groupby(ringed, :ring), :species => (s ->
  ↪ length(unique(s))) => :n_species,
  nrow => :n)
first(sort(per_ring[per_ring.n_species .> 1, :], :n, rev = true), 6)
```

| | ring
String31 | n_species
Int64 | n
Int64 |
|---|------------------|--------------------|------------|
| 1 | BRD | 25 | 31 |
| 2 | VRD | 24 | 27 |
| 3 | AZD | 21 | 26 |
| 4 | AZE | 22 | 24 |
| 5 | IND1 | 4 | 15 |
| 6 | BRE | 14 | 14 |

Some “rings” are shared by more than twenty species. They are not ring numbers but **colour-band codes** such as BRD or BLUE RIGHT, which many birds carry. A naive count of recaptures would take all those birds for one individual.

By source and method. Each record comes from one of 65 main researchers, and from live birds or museum skins, over two centuries:

```
combine(groupby(dropmissing(birds[:, [:status, :year]]), :status),
        nrow => :n, :year => minimum => :first, :year => median =>
  ↪ :median,
        :year => maximum => :last)
```

| | status | n | first | median | last |
|---|---------|-------|-------|---------|-------|
| | String7 | Int64 | Int64 | Float64 | Int64 |
| 1 | live | 57862 | 1990 | 2010.0 | 2018 |
| 2 | museum | 11152 | 1820 | 2007.0 | 2017 |

Each of these groupings can make records from the same group more alike than records from different groups — the dependence that the mixed models of 1 account for. Exploring the structure tells you which grouping terms a model needs.

i Your turn 7.3 — How many birds were caught twice?

Count the individuals recorded more than once: rings that appear in more than one record **of the same species**, ignoring ring codes made only of letters (colour-band codes). Pass the number to check("7.3", ...).

Hint

Drop the letter-only codes with `occursin(r"^[A-Za-z]+$"`, `ring`), which is true for them. Then group by ring **and** species, count rows, and count the groups with more than one.

Solution

```
real_rings = ringed[.!occursin(r"^[A-Za-z]+$",
  ↪ ringed.ring), :]
per_bird = combine(groupby(real_rings, [:ring,
  ↪ :species]), nrow => :n)
n_recaptured = count(>(1), per_bird.n)
```

4503

```
check("7.3", n_recaptured)
```

✓ Correct!

About 4 500 birds contribute two records or more, one of them up to 43. Treating their records as independent would overstate the sample size — and the colour codes would have added 27 false “individuals” to the count.

7.4 Distributions

Body mass across all birds spans three orders of magnitude, from hummingbirds to large seabirds. On a linear scale almost everything piles up at the left; on a log scale the shape becomes visible:

```
masses = collect(skipmissing(birds.mass_g))

fig = Figure(size = (860, 340))
ax1 = Axis(fig[1, 1], xlabel = "Body mass (g)", ylabel = "Records",
  ↪ title = "(a) Linear")
hist!(ax1, masses, bins = 60, color = (:steelblue, 0.7))
ax2 = Axis(fig[1, 2], xlabel = "Body mass (g, log scale)", title = "(b)
  ↪ Logarithmic",
  xscale = log10)
hist!(ax2, masses, bins = 10 .^ range(0, 3.6, length = 60), color =
  ↪ (:steelblue, 0.7))
fig
```

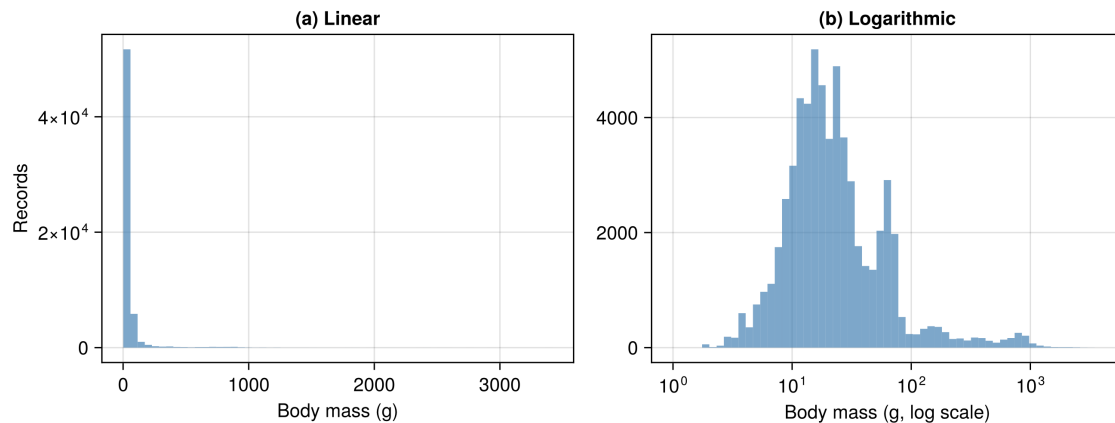


Figure 7.1: Body mass of all weighed birds, on a linear (a) and a logarithmic (b) scale.

Sizes, areas and concentrations are bounded at zero and grow multiplicatively, so they are usually right-skewed and closer to symmetric on a log scale. That is why models of such variables so often use their logarithm.

Within one species the picture is different. For the white-necked thrush (*Turdus albicollis*), one of the most recorded species, compare the distribution with a normal curve of the same mean and standard deviation:

```
thrush = collect(skipmissing(birds.mass_g[birds.species .== "Turdus
  ↪ albicollis"]))
μ, σ = mean(thrush), std(thrush)

fig = Figure(size = (860, 340))
```

```

ax1 = Axis(fig[1, 1], xlabel = "Body mass (g)", ylabel = "Density",
    ↪ title = "(a) Histogram")
hist!(ax1, thrush, bins = 40, normalization = :pdf, color = (:seagreen,
    ↪ 0.6))
xs = range(minimum(thrush), maximum(thrush), length = 200)
lines!(ax1, xs, @.(exp(-(xs - μ)^2 / (2σ^2)) / (σ * sqrt(2π))), color =
    ↪ :black)

ax2 = Axis(fig[1, 2], xlabel = "Normal quantiles", ylabel = "Body mass
    ↪ (g)",
    title = "(b) QQ plot")
n = length(thrush)
z = quantile.(Normal(), ((1:n) .- 0.5) ./ n)
scatter!(ax2, z, sort(thrush), color = (:seagreen, 0.5), markersize =
    ↪ 5)
ablines!(ax2, μ, σ, color = :black, linestyle = :dash)
fig

```

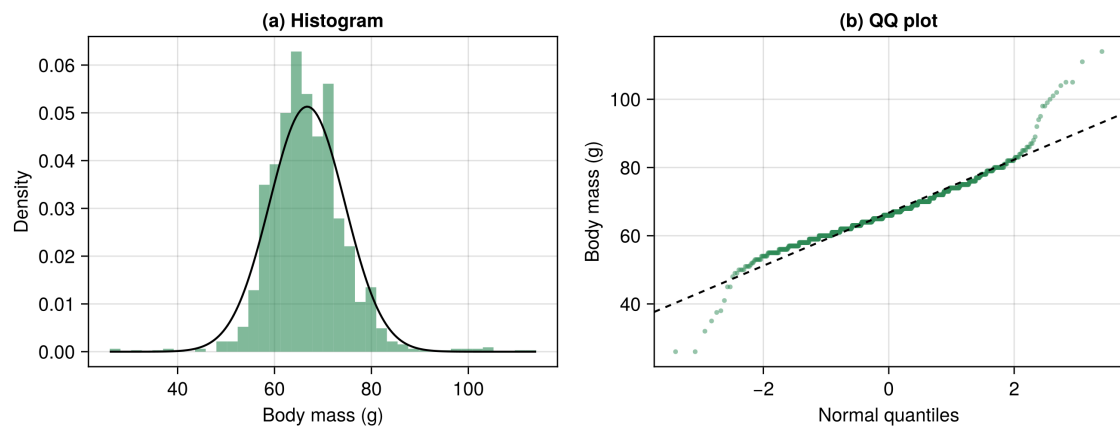


Figure 7.2: Body mass of *Turdus albicollis*: histogram with a normal curve of the same mean and standard deviation (a), and a QQ plot against the normal (b).

Close to normal in the middle, with a few birds far lighter or heavier than the rest — visible as the points that leave the line at both ends of the QQ plot. Those tails are the subject of the next section.

i Your turn 7.4 — Mean or median?

Compute the median and the mean body mass of *Turdus albicollis* and pass them, in that order, as a vector to `check("7.4", ...)`. Which is less affected by the extreme birds?

Hint

`thrush` above already holds the masses. `[median(x), mean(x)]` builds the vector.

Solution

```
thrush_centre = [median(thrush), mean(thrush)]
```

```
2-element Vector{Float64}:  
 66.0  
 66.74542818610924
```

```
check("7.4", thrush_centre)
```

✓ Correct!

66 and 66.7 g: nearly equal, as expected for a roughly symmetric distribution. The median ignores how extreme the tails are; the mean is pulled by them. With a few records at 26 g or 114 g, the mean moves slightly; with the gross errors of the next section, it would move much more.

7.5 Errors and outliers

An overall histogram cannot show an error in a 14 g bird recorded as 1 240 g: 1 240 g is a perfectly ordinary mass for a large bird. Outliers in trait data are outliers **relative to the species**. A simple, robust flag compares each mass with the median of its species:

```
weighed = dropmissing(birds[:, [:species, :mass_g, :age, :status,  
    ↪ :source]], :mass_g)  
weighed = transform(groupby(weighed, :species),  
    :mass_g => median => :species_median, nrow =>  
    ↪ :n_species)  
weighed.ratio = weighed.mass_g ./ weighed.species_median  
  
suspicious = sort(weighed[(weighed.n_species .>= 10) .& (weighed.ratio  
    ↪ .> 3), :],  
    :ratio, rev = true)  
first(suspicious[:, [:species, :mass_g, :species_median, :ratio, :age,  
    ↪ :status]], 8)
```

| | species | mass_g | species_median | ratio | age | status |
|---|------------------------|---------|----------------|---------|-----------|----------|
| | String31 | Float64 | Float64 | Float64 | String15? | String7? |
| 1 | Thlypopsis sordida | 1240.0 | 14.0 | 88.5714 | Unknown | museum |
| 2 | Thlypopsis sordida | 1070.0 | 14.0 | 76.4286 | Unknown | museum |
| 3 | Corythopsis delalandi | 214.0 | 15.0 | 14.2667 | Adult | live |
| 4 | Tityra cayana | 940.0 | 73.0 | 12.8767 | Juvenile | museum |
| 5 | Tityra cayana | 840.0 | 73.0 | 11.5068 | Juvenile | museum |
| 6 | Platyrinchus mystaceus | 102.0 | 9.0 | 11.3333 | missing | missing |
| 7 | Thalurania glaucopis | 50.0 | 4.5 | 11.1111 | Adult | live |
| 8 | Phaethornis eurynome | 55.0 | 5.0 | 11.0 | Adult | live |

The top of the list shows several kinds of problem:

- *Thlypopsis sordida*, a 14 g tanager, recorded at more than 1 kg: almost certainly a data-entry error, perhaps a misplaced decimal separator or a wrong unit.
- Several masses about **ten times** the species median — 102 g for a 9 g flycatcher, 50 g for a 4.5 g hummingbird: the signature of a shifted decimal point.
- Heavy juvenile *Tityra cayana* in a museum collection: possibly errors, possibly a mislabelled specimen. Only the source can tell.

Now the other tail:

```
first(sort(weighed[(weighed.n_species >= 10) & (weighed.ratio <
  ↪ 1/3), :], :ratio)[,
  [:species, :mass_g, :species_median, :ratio, :age, :status]], 6)
```

| | species | mass_g | species_median | ratio | age | status |
|---|---------------------------|---------|----------------|-----------|-----------|----------|
| | String31 | Float64 | Float64 | Float64 | String15? | String7? |
| 1 | Crypturellus parvirostris | 8.5 | 185.0 | 0.0459459 | Unknown | museum |
| 2 | Aramides saracura | 22.5 | 346.0 | 0.0650289 | Unknown | museum |
| 3 | Aramides saracura | 22.5 | 346.0 | 0.0650289 | Juvenile | museum |
| 4 | Aramides saracura | 24.0 | 346.0 | 0.0693642 | Unknown | museum |
| 5 | Aramides saracura | 24.0 | 346.0 | 0.0693642 | Juvenile | museum |
| 6 | Asio clamator | 34.5 | 470.0 | 0.0734043 | Unknown | museum |

Here some of the “outliers” are real. A 24 g slaty-breasted wood-rail (*Aramides saracura*, median 346 g) marked as juvenile is plausibly a chick. Deleting it would remove a real bird; keeping it in an analysis of adult size would distort it.

! Flag, don't delete

A statistical rule can only say that a value is unusual, never that it is wrong. Flag suspicious values, look at them one by one, go back to the source where you can, and decide — and write down what you decided and why, in code, so that the decision can be reviewed. For many analyses the right choice is to restrict the data to what the question is about (adults, say) rather than to remove individual points.

The data also carry a quieter problem, **precision**. Most masses are whole numbers:

```
mean(weighed.mass_g == round(weighed.mass_g))
```

0.7217974646080995

72% of masses are integers, which means many were weighed to the nearest gram. For a 3 g hummingbird, rounding to the gram is an error of up to 17%; for a thrush, under 1%. Precision that varies with the size of the animal is easy to miss and worth knowing before you compare small species.

i Your turn 7.5 — How many suspicious masses?

Count the records whose mass is more than 3 times, or less than a third of, the median of their species, among species with at least 10 weighed records. Pass the number to `check("7.5", ...)`.

Hint

`weighed` already has `ratio` and `n_species`. Combine the conditions with `.&` and `.|`, and count the trues with `count`.

Solution

```
n_suspicious = count((weighed.n_species .>= 10) .&
                      ((weighed.ratio .> 3) .|
                       (weighed.ratio .< 1/3)))
```

291

```
check("7.5", n_suspicious)
```

✓ Correct!

291 of about 60 000 weighed records, 0.5%: few, but enough to wreck a species mean. The threshold of 3 is a choice, not a rule; with 2 you would flag more than twice as many.

7.6 Relationships and collinearity

Morphological traits go together: big birds have long wings, long tails, long legs and long bills. To see how strongly, summarize each species by the median of each trait — the median, because of the outliers above — and compare species:

```
traits = [:mass_g, :wing_mm, :tail_mm, :tarsus_mm, :bill_mm]
median_or_missing(x) = (v = collect(skipmissing(x)); isempty(v) ?
  ↳ missing : median(v))

by_species = dropmissing(combine(groupby(birds, :species),
                                   [t => median_or_missing => t for t in
  ↳ traits]...))
log_traits = DataFrame(log.(Matrix(by_species[:, traits])), traits)
```

```
nrow(by_species)
```

538

```
round.(cor(Matrix(log_traits))); digits = 2)
```

5×5 Matrix{Float64}:

```
1.0  0.93  0.71  0.81  0.72
0.93  1.0  0.77  0.72  0.64
0.71  0.77  1.0  0.56  0.5
0.81  0.72  0.56  1.0  0.64
0.72  0.64  0.5  0.64  1.0
```

```
labels = ["log mass", "log wing", "log tail", "log tarsus", "log bill"]
fig = Figure(size = (820, 820))
for i in 1:5, j in 1:5
    ax = Axis(fig[i, j], xlabel = i == 5 ? labels[j] : "", ylabel = j
    ↪ == 1 ? labels[i] : "",
    ↪          xticklabelsvisible = i == 5, yticklabelsvisible = j == 1
    ↪ && i != 1)
    if i == j                # diagonal: the distribution of each
    ↪ trait (counts)
        hist!(ax, log_traits[:, i], bins = 25, color = (:grey60, 0.8))
    else
        scatter!(ax, log_traits[:, j], log_traits[:, i], markersize =
        ↪ 3,
        ↪          color = (:steelblue, 0.5))
    end
end
end
fig
```

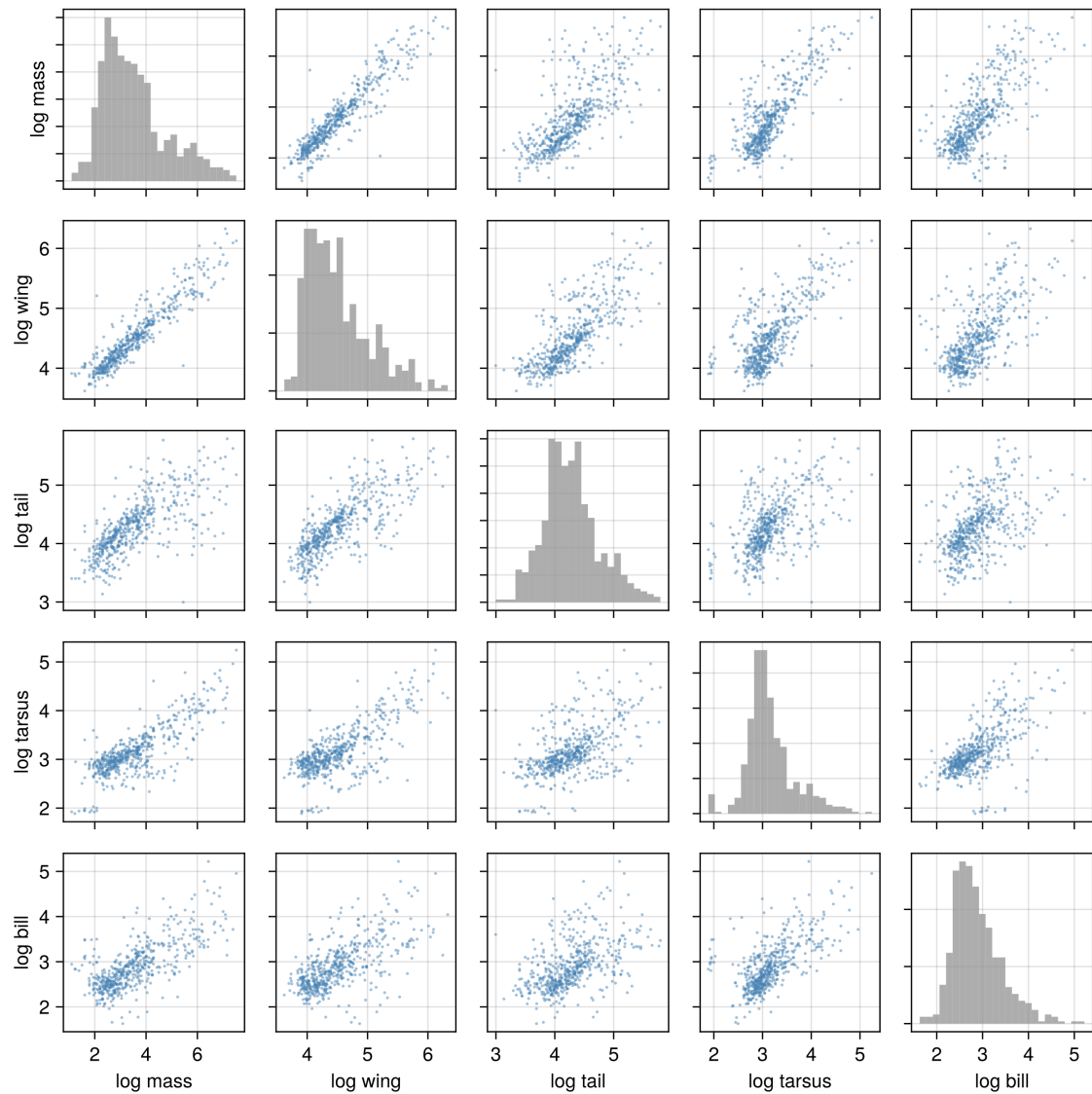


Figure 7.3: Species medians of five traits, on log scales, one point per species. Off the diagonal, each panel shows one pair of traits; on the diagonal, the distribution of each trait.

Mass and wing length are almost a single variable across species. If both entered a model as predictors, the model could not tell their effects apart: the estimates would be unstable and their standard errors large. The **variance inflation factor** measures this. For each predictor, regress it on all the others; with R^2 from that regression, $VIF = 1/(1 - R^2)$:

```
function vif(df, col)
  others = setdiff(propertynames(df), [col])
  f = Term(col) ~ sum(Term.(others))
```

```

    return 1 / (1 - r2(lm(f, df)))
end

DataFrame(trait = traits, vif = round([vif(log_traits, t) for t in
↪ traits]; digits = 1))

```

| | trait | vif |
|---|-----------|---------|
| | Symbol | Float64 |
| 1 | mass_g | 11.9 |
| 2 | wing_mm | 9.7 |
| 3 | tail_mm | 2.5 |
| 4 | tarsus_mm | 3.0 |
| 5 | bill_mm | 2.1 |

Zuur et al. (2010) use a cut-off of 3; other authors use 5 or 10. Mass and wing length are above all of them. The usual responses are to keep only one of each collinear group, to combine them (a size index), or to ask a question that does not need them separately.

i Your turn 7.6 — Mass and wing

What is the correlation between log body mass and log wing length across species, using the species medians above? Pass it to `check("7.6", ...)`.

Hint

`cor(x, y)` on two columns of `log_traits`.

Solution

```
r_mass_wing = cor(log_traits.mass_g, log_traits.wing_mm)
```

```
0.9326447165263585
```

```
check("7.6", r_mass_wing)
```

✓ Correct!

About 0.93. Across species, these two traits carry almost the same information. Within a species it is much weaker — about 0.25 among individual *Turdus albicollis* — because individuals of one species vary little in size: collinearity depends on the scale of the question.

7.7 Why the plots: the Anscombe quartet

Summaries are not a substitute for looking. Anscombe (1973) built four small data sets with the same means, variances, correlations and regression lines:

```
anscombe = dataset("datasets", "anscombe")
DataFrame(set = 1:4,
           mean_x = [mean(anscombe[:, "X$i"]) for i in 1:4],
           mean_y = [round(mean(anscombe[:, "Y$i"]); digits = 2) for i
↪   in 1:4],
           cor_xy = [round(cor(anscombe[:, "X$i"], anscombe[:, "Y$i"]);
↪   digits = 3) for i in 1:4])
```

| | set | mean_x | mean_y | cor_xy |
|---|-------|---------|---------|---------|
| | Int64 | Float64 | Float64 | Float64 |
| 1 | 1 | 9.0 | 7.5 | 0.816 |
| 2 | 2 | 9.0 | 7.5 | 0.816 |
| 3 | 3 | 9.0 | 7.5 | 0.816 |
| 4 | 4 | 9.0 | 7.5 | 0.817 |

```
fig = Figure(size = (760, 560))
for i in 1:4
    x, y = anscombe[:, "X$i"], anscombe[:, "Y$i"]
    ax = Axis(fig[(i - 1) ÷ 2 + 1, (i - 1) % 2 + 1], title = "Set $i",
              limits = (3, 20, 3, 14))
    scatter!(ax, x, y, color = :steelblue)
    b = coef(lm(@formula(y ~ x), DataFrame(; x, y)))
    ablines!(ax, b[1], b[2], color = :darkorange)
end
fig
```

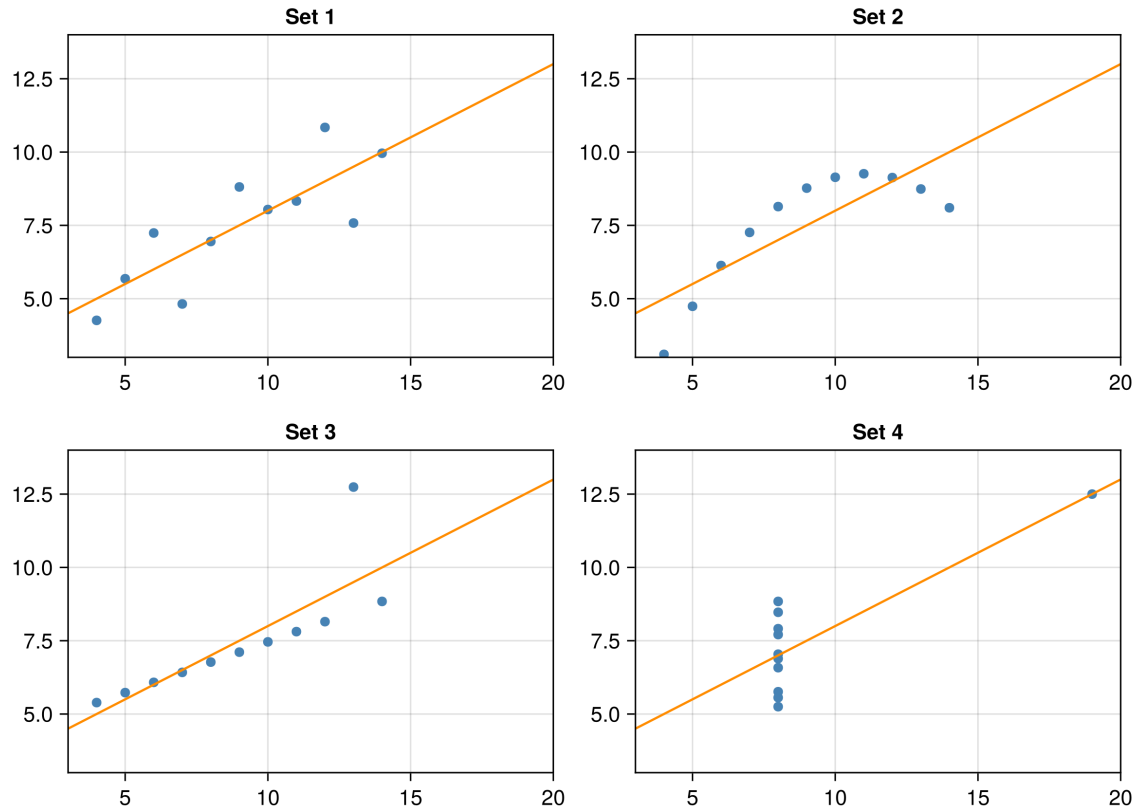


Figure 7.4: The Anscombe quartet: four data sets with the same summary statistics and the same least-squares line.

The first looks like what a linear model assumes. The second is a curve. In the third one point pulls the line away from an otherwise perfect relationship. In the fourth the whole slope depends on a single observation. Only the plots show it.

Challenges

1. Plot the coordinates of all records (lon, lat) and colour them by `outside_range`. Are the records flagged as outside the species' range where you would expect? Are there points outside the Atlantic forest region altogether?
2. Male and female *Chiroxiphia caudata* (blue manakin) differ strikingly in plumage. Do they differ in mass? Compare the distributions, not only the medians, and decide what to do with records of "Unknown" sex.
3. Repeat the outlier rule of Section 7.5 for wing length. Are the species with suspicious wings the same as those with suspicious masses? What would that tell you about the source of the errors?

Part III

Modelling

8 Linear models

```
using TidierData
using DataFrames, CSV
using GLM, StatsModels, StatsBase, Statistics, Distributions
using HypothesisTests
using CairoMakie
using MixedModels

CairoMakie.activate!(type = "png")

ponds = @chain CSV.read(joinpath("../", "data", "ponds.csv"), DataFrame)
↳ begin
    @drop_missing(canopy)
    @mutate(log_area = log(area_m2))
end

nrow(ponds)
```

173

i Exercises in this chapter

In the *Your turn* boxes you fit a model and pass a number extracted from it — a p-value, coefficients, an interval — to check. All of them use the ponds with canopy recorded, as in this chapter. Start each session with:

```
include("exercises/check.jl")
using TidierData, DataFrames, CSV, GLM, StatsModels, Statistics
using HypothesisTests, MixedModels
ponds = @chain CSV.read(joinpath("data", "ponds.csv"), DataFrame)
↳ begin
    @drop_missing(canopy)
    @mutate(log_area = log(area_m2))
end
```

8.1 Hypothesis tests

Before models, the basics. `HypothesisTests.jl` covers the usual repertoire.

```
perm = @chain ponds begin @filter(hydro == "permanent") end
temp_ = @chain ponds begin @filter(hydro == "temporary") end

UnequalVarianceTTest(perm.richness, temp_.richness)
```

Two sample t-test (unequal variance)

Population details:

```
parameter of interest: Mean difference
value under h_0:      0
point estimate:       1.01849
95% confidence interval: (-0.452, 2.489)
```

Test summary:

```
outcome with 95% confidence: fail to reject h_0
two-sided p-value:          0.1734
```

Details:

```
number of observations: [95,78]
t-statistic:            1.3673234271337709
degrees of freedom:     168.0376473521344
empirical standard error: 0.7448775533304762
```

In R

`t.test(richness ~ hydro, data = ponds)` uses Welch by default — the same as `UnequalVarianceTTest`. For the equal-variance test, use `EqualVarianceTTest`.

i Your turn 8.1 — Does hydroperiod affect body size?

Compare `body_size` between permanent and temporary ponds with a Welch t test. Pass its p-value to `check("8.1", ...)`.

Hint

Reuse `perm` and `temp_` from the block above. `pvalue(test)` extracts the p-value.

Solution

```
t_body = UnequalVarianceTTest(perm.body_size,
  ↪ temp_.body_size)
pvalue(t_body)
```

```
0.10269215805061378
```

```
check("8.1", pvalue(t_body))
```

✓ Correct!

$p \approx 0.10$: no evidence of a difference. Here we can go further than usual, because in the simulation hydroperiod really has no effect on body size. With real data, a non-significant test does not show that there is no effect — only that the data cannot tell.

```
# normality and correlation
(
  ks_normal = ExactOneSampleKSTest(
    (ponds.body_size .- mean(ponds.body_size)) ./
    ↪ std(ponds.body_size),
    ↪ Normal(0, 1)) |> pvalue,
    correlation = CorrelationTest(ponds.canopy, ponds.body_size) |>
    ↪ pvalue,
  )
)
```

```
(ks_normal = 0.9802464726099859, correlation = 4.331805547836094e-5)
```

8.2 Simple linear regression

The dataset's continuous response is `body_size`, mean body size.

```
m1 = lm(@formula(body_size ~ temp_c), ponds)
```

```
StatsModels.TableRegressionModel{LinearModel{GLM.LmResp{Vector{Float64}}, GLM.DensePredCho
```

```
body_size ~ 1 + temp_c
```

Coefficients:

| | Coef. | Std. Error | t | Pr(> t) | Lower 95% | Upper 95% |
|-------------|----------|------------|-------|----------|-----------|-----------|
| (Intercept) | 13.1837 | 1.5861 | 8.31 | <1e-13 | 10.0528 | 16.3145 |
| temp_c | 0.921736 | 0.0645072 | 14.29 | <1e-30 | 0.794403 | 1.04907 |

The formula interface of `StatsModels.jl` matches R's: `~`, `+`, `*`, `:`, `-1` and inline transformations such as `log(x)` all work as expected.

```
(
  r2      = r2(m1),
  adj_r2  = adjr2(m1),
  aic     = aic(m1),
  coefs   = coef(m1),
  intervals = confint(m1),
)
```

```
(r2 = 0.544209635943369, adj_r2 = 0.5415441952178917, aic = 801.3969591368274, coefs = [13
```

i Your turn 8.2 — Body size and canopy

Fit a simple regression of `body_size` on `canopy` and pass both coefficients to `check("8.2", ...)`.

Hint

`coef(m)` returns the intercept and the slope, in that order.

Solution

```
m_canopy = lm(@formula(body_size ~ canopy), ponds)
coef(m_canopy)
```

```
2-element Vector{Float64}:
 38.13443532010066
 -0.049665628867203146
```

```
check("8.2", coef(m_canopy))
```

✓ Correct!

The slope, about -0.050 , is further from the true -0.04 than in `m2` (next section), because this model leaves temperature out.

8.3 Multiple regression

```
m2 = lm(@formula(body_size ~ temp_c + canopy), ponds)
```

```
StatsModels.TableRegressionModel{LinearModel{GLM.LmResp{Vector{Float64}}, GLM.DensePredChol{
```

```
body_size ~ 1 + temp_c + canopy
```

Coefficients:

| | Coef. | Std. Error | t | Pr(> t) | Lower 95% | Upper 95% |
|-------------|------------|------------|-------|----------|-----------|------------|
| (Intercept) | 15.761 | 1.54229 | 10.22 | <1e-18 | 12.7164 | 18.8055 |
| temp_c | 0.901451 | 0.059815 | 15.07 | <1e-32 | 0.783375 | 1.01953 |
| canopy | -0.0423641 | 0.00777948 | -5.45 | <1e-06 | -0.057721 | -0.0270073 |

The generating process (see `scripts/make_data.jl`) used $38.0 + 0.9(\text{temp} - 24.5) - 0.04 \text{ canopy}$. Compare with the estimates: this is the payoff of teaching with simulated data.

```
DataFrame(
  term      = coefnames(m2),
  estimated = round.(coef(m2), digits = 3),
  truth     = [38.0 - 0.9 * 24.5, 0.9, -0.04],
)
```

| | term | estimated | truth |
|---|-------------|-----------|---------|
| | String | Float64 | Float64 |
| 1 | (Intercept) | 15.761 | 15.95 |
| 2 | temp_c | 0.901 | 0.9 |
| 3 | canopy | -0.042 | -0.04 |

i Your turn 8.3 — Does the interval catch the truth?

Fit `body_size ~ temp_c + canopy + hydro` and compute the 95% confidence interval for the `temp_c` coefficient. Does it contain the true value, 0.9? Pass the interval (two numbers) to `check("8.3", ...)`.

Hint

`confint(m)` returns one row per coefficient; `temp_c` is the second row, so `confint(m)[2, :]`.

Solution

```
m_hydro = lm(@formula(body_size ~ temp_c + canopy +
  ↪ hydro), ponds)
ci_temp = confint(m_hydro)[2, :]
```

```
2-element Vector{Float64}:
 0.7785892869884911
 1.0167713699307872
```

```
check("8.3", ci_temp)
```

✓ Correct!

The interval, about $[0.78, 1.02]$, contains 0.9. It would not in about one of every 20 simulated datasets: that is what “95%” means.

8.4 Categorical predictors and interactions

```
m3 = lm(@formula(body_size ~ temp_c + canopy + region), ponds)
```

```
StatsModels.TableRegressionModel{LinearModel{GLM.LmResp{Vector{Float64}}, GLM.DensePredCho
```

```
body_size ~ 1 + temp_c + canopy + region
```

Coefficients:

| | Coef. | Std. Error | t | Pr(> t) | Lower 95% | Upper 95% |
|------------------|------------|------------|-------|----------|-----------|-----------|
| (Intercept) | 15.769 | 1.54634 | 10.20 | <1e-18 | 12.7162 | 18.8218 |
| temp_c | 0.893539 | 0.0597492 | 14.95 | <1e-31 | 0.775583 | 1.0115 |
| canopy | -0.0437397 | 0.00780336 | -5.61 | <1e-07 | -0.059145 | - |
| 0.0283344 | | | | | | |
| region: Atlantic | 0.664579 | 0.436092 | 1.52 | 0.1294 | -0.196347 | 1.52551 |
| region: Cerrado | 0.018937 | 0.43474 | 0.04 | 0.9653 | -0.83932 | 0.877194 |

Julia treats String and CategoricalArray columns as factors automatically, with treatment (dummy) coding and the first level as reference — same as R. To change the reference level or the contrast scheme:

```
m3b = lm(@formula(body_size ~ temp_c + region), ponds;  
         contrasts = Dict(:region => DummyCoding(base = "Cerrado")))  
  
coefnames(m3b)
```

```
4-element Vector{String}:
```

```
"(Intercept)"
```

```
"temp_c"
```

```
"region: Amazon"
```

```
"region: Atlantic"
```

i Your turn 8.4 — Choosing the reference

Fit `body_size ~ temp_c + region` with the Atlantic region as the reference level and pass the coefficient names to check("8.4", ...).

Hint

Copy the contrasts argument from the block above and change the level.

Solution

```
m_atl = lm(@formula(body_size ~ temp_c + region), ponds;
           contrasts = Dict(:region => DummyCoding(base =
             ↪ "Atlantic"))))
coefnames(m_atl)
```

```
4-element Vector{String}:
 "(Intercept)"
 "temp_c"
 "region: Amazon"
 "region: Cerrado"
```

```
check("8.4", coefnames(m_atl))
```

✓ Correct!

The reference level has no coefficient of its own: it is absorbed by the intercept, and the other regions are compared with it. Without contrasts, the reference is the first level in alphabetical order, Amazon.

```
m4 = lm(@formula(body_size ~ temp_c * hydro), ponds)
```

```
StatsModels.TableRegressionModel{LinearModel{GLM.LmResp{Vector{Float64}}, GLM.DensePredChol{Vector{Float64}}}}
```

```
body_size ~ 1 + temp_c + hydro + temp_c & hydro
```

Coefficients:

| | Coef. | Std. Error | t | Pr(> t) | Lower 95% | Upper 95% |
|---------------------------|----------|------------|-------|----------|-----------|-----------|
| (Intercept) | 14.8606 | 2.16225 | 6.87 | <1e-09 | 10.5921 | 19.1291 |
| temp_c | 0.847037 | 0.0890498 | 9.51 | <1e-16 | 0.671244 | 1.02283 |
| hydro: temporary | -3.33192 | 3.20923 | -1.04 | 0.3006 | -9.66726 | 3.00341 |
| temp_c & hydro: temporary | 0.147595 | 0.130228 | 1.13 | 0.2587 | -0.109489 | 0.404679 |

8.5 Model selection and comparison

```
models = ["temp"      => m1,
          "temp+canopy" => m2,
          "temp+can+reg" => m3,
          "temp*hydro"  => m4]
```

```
DataFrame(
  model = first.(models),
  k     = [length(coef(m)) for (_, m) in models],
  aic   = round.([aic(m) for (_, m) in models], digits = 1),
  bic   = round.([bic(m) for (_, m) in models], digits = 1),
  adj_r2 = round.([adjr2(m) for (_, m) in models], digits = 3),
)
```

| | model | k | aic | bic | adj_r2 |
|---|--------------|-------|---------|---------|---------|
| | String | Int64 | Float64 | Float64 | Float64 |
| 1 | temp | 2 | 801.4 | 810.9 | 0.542 |
| 2 | temp+canopy | 3 | 775.6 | 788.2 | 0.607 |
| 3 | temp+can+reg | 5 | 776.1 | 795.0 | 0.611 |
| 4 | temp*hydro | 4 | 803.5 | 819.3 | 0.541 |

```
# F test between nested models, equivalent to anova() in R
fctest(m1.model, m2.model, m3.model)
```

F-test: 3 models fitted on 173 observations

| | DOF | ΔDOF | SSR | ΔSSR | R ² | ΔR ² | F* | p(>F) |
|-----|-----|------|-----------|-----------|----------------|-----------------|---------|--------|
| [1] | 3 | | 1005.3438 | | 0.5442 | | | |
| [2] | 4 | 1 | 856.0194 | -149.3243 | 0.6119 | 0.0677 | 29.6549 | <1e-06 |
| [3] | 6 | 2 | 839.0138 | -17.0056 | 0.6196 | 0.0077 | 1.7026 | 0.1853 |

8.6 Diagnostics

```
res = residuals(m3)
fit_ = predict(m3)

fig = Figure(size = (860, 350))

ax1 = Axis(fig[1, 1], xlabel = "Fitted values", ylabel = "Residuals",
           title = "(a) Residuals vs fitted")
scatter!(ax1, fit_, res, color = (:steelblue, 0.6))
hlines!(ax1, 0, color = :grey40, linestyle = :dash)

ax2 = Axis(fig[1, 2], xlabel = "Theoretical quantiles", ylabel =
  ↳ "Sample quantiles",
           title = "(b) QQ plot")
# NOTE: Makie exports `qqnorm` as a plot recipe, so the quantiles are
# computed explicitly here rather than taken from a helper.
n = length(res)
theoretical = quantile.(Normal(0, 1), ((1:n) .- 0.5) ./ n)
scatter!(ax2, theoretical, sort(res), color = (:darkorange, 0.6))
```

```
ablines!(ax2, 0, std(res), color = :grey40, linestyle = :dash)

ax3 = Axis(fig[1, 3], xlabel = "Residuals", ylabel = "Frequency",
           title = "(c) Distribution")
hist!(ax3, res, bins = 20, color = (:seagreen, 0.7))

fig
```

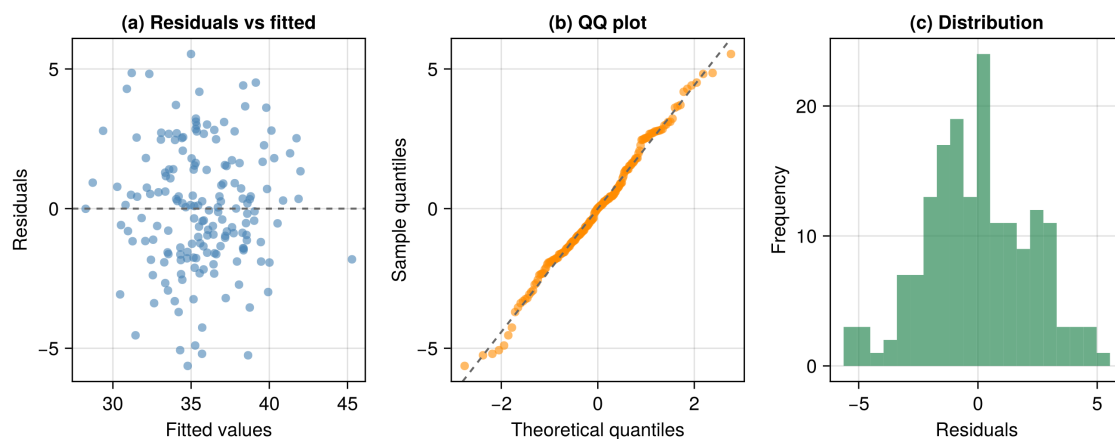


Figure 8.1: Residual diagnostics for model m3.

⚠ A real gap

R has `plot(model)`, which returns the four diagnostic plots at once, and `performance::check_model()`, which goes much further. Julia has no consolidated equivalent; the diagnostics above are assembled by hand. This is a genuine gap, and exactly the kind of thing a community package would fix well.

i Your turn 8.5 — How many large residuals?

If the residuals of `m3` are roughly normal, about 5% of them should lie more than two standard deviations from zero. Compute the proportion that does and pass it to `check("8.5", ...)`.

Hint

Divide `res` (from the diagnostics block) by `std(res)`, then take the mean of `abs.(z) .> 2` — the mean of true/false values is a proportion.

Solution

```
z = res ./ std(res)
prop_large = mean(abs.(z) .> 2)
```

0.057803468208092484

```
check("8.5", prop_large)
```

✓ Correct!

About 5.8% — 10 of 173 residuals, close to the 5% expected. This is a numerical counterpart to the QQ plot above.

8.7 Mixed models

MixedModels.jl (Bates et al. 2026) is maintained by Douglas Bates, first author of lme4 (Bates et al. 2015), and uses the same formula notation — including $(1 \mid \text{group})$.

On performance: comparisons between the two depend heavily on the model, the size of the data and the random-effects structure, and I know of no published benchmark I can cite here. If fitting time matters for your problem, measure it on your own case.

```
mm1 = fit(MixedModel,  
          @formula(body_size ~ temp_c + canopy + (1 | region)),  
          ponds)
```

Minimizing 2 Time: 0:00:00 (95.50 ms/it)
objective: 780.4261079240766

Minimizing 19 Time: 0:00:00 (19.58 ms/it)

| | Est. | SE | z | p | σ_{region} |
|-------------|---------|--------|-------|--------|--------------------------|
| (Intercept) | 15.7973 | 1.5290 | 10.33 | <1e-24 | 0.1246 |
| temp_c | 0.9002 | 0.0592 | 15.20 | <1e-51 | |
| canopy | -0.0426 | 0.0077 | -5.52 | <1e-07 | |
| Residual | 2.2209 | | | | |

```
(  
  fixed_effects = fixef(mm1),  
  variance      = VarCorr(mm1),  
  icc           = first(mm1.σs.region)^2 /  
                  (first(mm1.σs.region)^2 + mm1.σ^2),  
)
```

```
(fixed_effects = [15.79726349296374, 0.9002362212634433, -0.04256924166049193], variance =
      Column    Variance Std.Dev.
region  (Intercept)  0.015537 0.124649
Residual                4.932481 2.220919
, icc = 0.0031401446304593875)
```

In R

```
lmer(body_size ~ temp_c + canopy + (1 | region), data = ponds)
```

Identical, down to the (1 | group) notation. MixedModels.jl also supports (1 + x | group), (1 | a) + (1 | b) and nested terms.

i Your turn 8.6 — A random intercept for region

Fit a mixed model of body_size on temp_c with a random intercept for region and pass its fixed effects to check("8.6", ...). Then look at VarCorr of your model: how much variation does region account for?

Hint

Same structure as mm1, with a shorter formula. fixef(mm) returns the fixed effects.

Solution

```
mm_temp = fit(MixedModel, @formula(body_size ~ temp_c +
  ⇨ (1 | region)), ponds;
              progress = false)
fixef(mm_temp)
```

```
2-element Vector{Float64}:
 13.183682687070245
  0.9217361109462586
```

```
check("8.6", fixef(mm_temp))
VarCorr(mm_temp)
```

✓ Correct!

| | Column | Variance | Std.Dev |
|----------|-------------|----------|---------|
| region | (Intercept) | 0.00000 | 0.00000 |
| Residual | | 5.81124 | 2.41065 |

The region variance is estimated as exactly zero — a *singular* fit. The simulation has no region effect on body size, and three regions give very little information about a variance anyway. With a zero variance, the mixed model reduces to `lm(@formula(body_size ~ temp_c), ponds)`: compare the coefficients.

```
# parametric bootstrap intervals – fast enough to be routine
using Random
boot = parametricbootstrap(MersenneTwister(42), 1000, mm1)
DataFrame(shortestcovint(boot))
```

Challenges

1. Fit `body_size ~ canopy` separately for each region and compare the slopes with those of a single model with a `canopy * region` interaction. Why do they agree?
2. Does `m4` have a significant interaction? The simulation has no `temp_c × hydro` interaction: if yours were significant, what would that mean?
3. In 8.6 the region variance came out as zero. Compare `mm1` with a model that treats region as a fixed effect. With three regions, which would you report, and why?

9 Generalized Linear Models

```
using TidierData
using DataFrames, CSV
using GLM, StatsModels, StatsBase, Statistics, Distributions
using MixedModels
using CairoMakie

CairoMakie.activate!(type = "png")

ponds = @chain CSV.read(joinpath("../", "data", "ponds.csv"), DataFrame)
↳ begin
    @drop_missing(canopy)
    @mutate(log_area = log(area_m2))
end

nrow(ponds)
```

173

i Exercises in this chapter

As in Chapter 8, you fit a model in each *Your turn* box and pass a number extracted from it to check. Start each session with:

```
include("exercises/check.jl")
using TidierData, DataFrames, CSV, GLM, StatsModels, Statistics,
↳ MixedModels
ponds = @chain CSV.read(joinpath("data", "ponds.csv"), DataFrame)
↳ begin
    @drop_missing(canopy)
    @mutate(log_area = log(area_m2))
end
```

9.1 Why GLMs in ecology

Counts are not normal, proportions live in $[0, 1]$, presences are binary. The Gaussian linear model treats all of them as if they were continuous and unbounded, and fails in predictable ways: confidence intervals that include negative richness, misspecified variance, fan-shaped residuals.

The GLM fixes this with two choices: the **distribution** of the response and the **link function** connecting the mean to the linear predictor.

| Response | Distribution | Link | In Julia |
|-----------------------------|--------------------|-------|----------------------------|
| Count | Poisson | log | Poisson(),
LogLink() |
| Overdispersed count | Negative binomial | log | negbin(...) |
| Presence/absence | Bernoulli/Binomial | logit | Binomial(),
LogitLink() |
| Proportion with denominator | Binomial | logit | Binomial(),
LogitLink() |
| Positive skewed continuous | Gamma | log | Gamma(),
LogLink() |

9.2 Poisson: species richness

```
g1 = glm(@formula(richness ~ log_area + canopy + temp_c),
         ponds, Poisson(), LogLink())
```

StatsModels.TableRegressionModel{GeneralizedLinearModel{GLM.GlmResp{Vector{Float64}}, Poisson{}}

richness ~ 1 + log_area + canopy + temp_c

Coefficients:

| | Coef. | Std. Error | z | Pr(> z) | Lower 95% | Upper 95% |
|-------------|-----------|------------|-------|----------|------------|-------------|
| (Intercept) | -0.30001 | 0.289756 | -1.04 | 0.3005 | -0.867921 | 0.267902 |
| log_area | 0.331763 | 0.0300907 | 11.03 | <1e-27 | 0.272786 | 0.390739 |
| canopy | -0.011555 | 0.0013362 | -8.65 | <1e-17 | -0.0141739 | -0.00893614 |
| temp_c | 0.0441958 | 0.00959497 | 4.61 | <1e-05 | 0.02539 | 0.0630016 |

The generating process used $\log \mu = 0.9 + 0.28 \log(\text{area}) - 0.011 \text{ canopy} + 0.03 (\text{temp} - 24.5) + \dots$

```
DataFrame(
  term      = coefnames(g1),
  estimated = round.(coef(g1), digits = 4),
  truth     = [missing, 0.28, -0.011, 0.03],
)
```

| | term | estimated | truth |
|---|-------------|-----------|----------------|
| | String | Float64 | Float64? |
| 1 | (Intercept) | -0.3 | <i>missing</i> |
| 2 | log_area | 0.3318 | 0.28 |
| 3 | canopy | -0.0116 | -0.011 |
| 4 | temp_c | 0.0442 | 0.03 |

i Your turn 9.1 — Adding hydroperiod

Fit a Poisson GLM like `g1`, adding `hydro` at the end of the formula. Pass its coefficients to `check("9.1", ...)`, then compare its AIC with that of `g1`.

Hint

Copy `g1` and add `+ hydro`. `aic(g)` gives the AIC; lower is better.

Solution

```
g_hydro = glm(@formula(richness ~ log_area + canopy +
  ↪ temp_c + hydro),
              ponds, Poisson(), LogLink())
coef(g_hydro)
```

5-element Vector{Float64}:

```
-0.29558194444246727
 0.3308987450570612
-0.011543093000739657
 0.047527176727807284
-0.18958052468264142
```

```
check("9.1", coef(g_hydro))
(aic_g1 = aic(g1), aic_hydro = aic(g_hydro))
```

✓ Correct!

```
(aic_g1 = 952.9021516391434, aic_hydro = 944.1070557908266)
```

The AIC drops by about 9, so `hydro` earns its place. The simulation added 0.22 to the log mean of permanent ponds; the model estimates -0.19 for temporary ones, the same effect seen from the other side.

💡 Interpreting coefficients on the log scale

A coefficient β under a log link means that a one-unit increase in the predictor multiplies the mean by e^β . For canopy:

```
exp(coef(g1)[3]), (exp(coef(g1)[3]) - 1) * 100
```

```
(0.9885114593513831, -1.148854064861693)
```

That is, each additional percentage point of canopy reduces expected richness by about 1%.

i Your turn 9.2 — Temporary ponds, in plain numbers

Using the model from 9.1, by what factor does expected richness change in a temporary pond compared with a permanent one, all else equal? Pass that factor to `check("9.2", ...)`.

Hint

The hydro: temporary coefficient is the fifth. Under a log link, a coefficient β multiplies the mean by e^β .

Solution

```
factor_temp = exp(coef(g_hydro)[5])
```

```
0.8273060956543651
```

```
check("9.2", factor_temp)
```

✓ Correct!

About 0.83: temporary ponds are expected to hold about 17% fewer species than permanent ones.

9.3 Overdispersion

The Poisson assumes $\text{Var}(Y) = E(Y)$. Ecological data rarely cooperate.

```
# Pearson residual: (y - mu) / sqrt(Var(mu)). For the Poisson, Var(mu)
↪ = mu.
mu = predict(g1)
pearson_res = (ponds.richness - mu) ./ sqrt.(mu)
df_res = dof_residual(g1)
phi = sum(pearson_res .^ 2) / df_res

(dispersion_statistic = phi,
```

```
verdict = phi > 1.5 ? "overdispersed" : "acceptable")
```

```
(dispersion_statistic = 1.9527034194568558, verdict = "overdispersed")
```

If $\hat{\phi} \gg 1$, the alternatives are the negative binomial or a mixed model with an observation-level random effect.

! Overdispersion is a symptom, not a diagnosis

ponds.csv was generated as a *pure* Poisson process — there is no extra-Poisson variance in the data-generating code. Yet $\hat{\phi} \approx 1.95$. Why?

Because the generating model included a region effect that g1 omits. Unmodelled heterogeneity inflates the dispersion statistic exactly as genuine overdispersion does, and the statistic cannot tell you which one you have.

This matters practically: the reflex of reaching for a negative binomial whenever $\hat{\phi} > 1.5$ can paper over a missing predictor. Check your model structure before you change your distribution. The GLMM in Section 9.6 addresses the actual cause here.

i Your turn 9.3 — Testing the warning

The box above claims that the overdispersion of g1 comes from a missing predictor. Test it: fit a Poisson GLM with log_area, canopy, temp_c, hydro and region, compute its dispersion statistic as in the block above, and pass it to check("9.3", ...).

Hint

Copy the dispersion block, replacing g1 with your model. predict(g) gives the fitted means.

Solution

```
g_full = glm(@formula(richness ~ log_area + canopy +
  ↪ temp_c + hydro + region),
  ponds, Poisson(), LogLink())

mu_full = predict(g_full)
phi_full = sum(((ponds.richness .- mu_full) ./
  ↪ sqrt(mu_full)).^2) /
  dof_residual(g_full)
```

```
0.9410041742922787
```

```
check("9.3", phi_full)
```

✓ Correct!

About 0.94: with the predictors of the generating model in place, the “overdispersion” is gone. Adding hydro alone (9.1) barely moved it; region was the missing piece.

```
g2 = negbin(@formula(richness ~ log_area + canopy + temp_c), ponds,
  ↪ LogLink())
```

StatsModels.TableRegressionModel{GeneralizedLinearModel{GLM.GlmResp{Vector{Float64}}, Negat.

richness ~ 1 + log_area + canopy + temp_c

Coefficients:

| | Coef. | Std. Error | z | Pr(> z) | Lower 95% | Upper 95% |
|-------------|------------|------------|-------|----------|------------|-------------|
| (Intercept) | -0.165087 | 0.424739 | -0.39 | 0.6975 | -0.997561 | 0.667387 |
| log_area | 0.334165 | 0.0432899 | 7.72 | <1e-13 | 0.249319 | 0.419012 |
| canopy | -0.0120529 | 0.0019084 | -6.32 | <1e-09 | -0.0157933 | -0.00831254 |
| temp_c | 0.0391606 | 0.0141787 | 2.76 | 0.0057 | 0.0113708 | 0.0669503 |

```
DataFrame(
  model = ["Poisson", "Negative binomial"],
  aic = round([aic(g1), aic(g2)], digits = 1),
  bic = round([bic(g1), bic(g2)], digits = 1),
  loglik = round([loglikelihood(g1), loglikelihood(g2)], digits =
  ↪ 1),
)
```

| | model | aic | bic | loglik |
|---|-------------------|---------|---------|---------|
| | String | Float64 | Float64 | Float64 |
| 1 | Poisson | 952.9 | 965.5 | -472.5 |
| 2 | Negative binomial | 911.3 | 927.0 | -450.6 |

9.4 Offsets

When sampling effort varies across units, model the **rate** rather than the raw count, using an offset.

```
g3 = glm(@formula(richness ~ canopy + temp_c), ponds, Poisson(),
  ↪ LogLink();
  offset = log.(ponds.area_m2))

coefstable(g3)
```

| | Coef. | Std. Error | z | Pr(> z) | Lower 95% | Upper 95% |
|-------------|-----------|------------|--------|----------|-----------|-----------|
| (Intercept) | -3.56359 | 0.244593 | -14.57 | <1e-47 | -4.04299 | -3.0842 |
| canopy | - | 0.001304 | -11.60 | <1e-30 | - | - |
| | 0.0151224 | | | | 0.0176782 | 0.0125666 |
| temp_c | 0.0310744 | 0.00949171 | 3.27 | 0.0011 | 0.012471 | 0.0496778 |

This models richness *per unit area*, fixing the $\log(\text{area})$ coefficient at 1.

9.5 Logistic regression

```
g4 = glm(@formula(focal ~ canopy + temp_c + hydro),
        ponds, Binomial(), LogitLink())
```

StatsModels.TableRegressionModel{GeneralizedLinearModel{GLM.GlmResp{Vector{Float64}}, Binom.

focal ~ 1 + canopy + temp_c + hydro

Coefficients:

| | Coef. | Std. Error | z | Pr(> z) | Lower 95% | Upper 95% |
|------------------|-----------|------------|-------|----------|-----------|-----------|
| (Intercept) | -5.79176 | 1.6009 | -3.62 | 0.0003 | -8.92947 | - |
| 2.65406 | | | | | | |
| canopy | 0.0172429 | 0.00759302 | 2.27 | 0.0232 | 0.0023609 | 0.032125 |
| temp_c | 0.209118 | 0.0616577 | 3.39 | 0.0007 | 0.088271 | 0.329965 |
| hydro: temporary | -0.926093 | 0.338929 | -2.73 | 0.0063 | -1.59038 | - |
| 0.261805 | | | | | | |

```
DataFrame(
  term      = coefnames(g4),
  coef      = round.(coef(g4), digits = 4),
  odds_ratio = round.(exp.(coef(g4)), digits = 3),
  ci_lower   = round.(exp.(confint(g4)[:, 1]), digits = 3),
  ci_upper   = round.(exp.(confint(g4)[:, 2]), digits = 3),
)
```

| | term | coef | odds_ratio | ci_lower | ci_upper |
|---|------------------|---------|------------|----------|----------|
| | String | Float64 | Float64 | Float64 | Float64 |
| 1 | (Intercept) | -5.7918 | 0.003 | 0.0 | 0.07 |
| 2 | canopy | 0.0172 | 1.017 | 1.002 | 1.033 |
| 3 | temp_c | 0.2091 | 1.233 | 1.092 | 1.391 |
| 4 | hydro: temporary | -0.9261 | 0.396 | 0.204 | 0.77 |

i Your turn 9.4 — Odds for a meaningful change

The odds ratio for one percentage point of canopy is close to 1, which is hard to interpret. Compute the odds ratio for an increase of **10** percentage points, from g4, and pass it to `check("9.4", ...)`.

Hint

Ten points multiply the odds by $e^{10\beta}$, not by $10 e^\beta$.

Solution

```
or_10 = exp(10 * coef(g4)[2])
```

```
1.1881878414800107
```

```
check("9.4", or_10)
```

✓ Correct!

About 1.19: each extra 10 points of canopy multiply the odds of finding the focal species by 1.19, an increase of 19%.

```
grid = DataFrame(
  canopy = repeat(range(0, 100, length = 100), 2),
  temp_c = fill(mean(ponds.temp_c), 200),
  hydro = repeat(["permanent", "temporary"], inner = 100),
)
grid.p = predict(g4, grid)

fig = Figure(size = (700, 430))
ax = Axis(fig[1, 1], xlabel = "Canopy cover (%)",
          ylabel = "P(occurrence)", title = "Focal species")

for (h, col) in zip(["permanent", "temporary"], [:steelblue,
  ↪ :darkorange])
  sub = grid[grid.hydro .== h, :]
  lines!(ax, sub.canopy, sub.p, color = col, linewidth = 2.5, label
  ↪ = h)
end
scatter!(ax, ponds.canopy, Float64.(ponds.focal), color = (:grey40,
  ↪ 0.25))
axislegend(ax, position = :lt)

fig
```

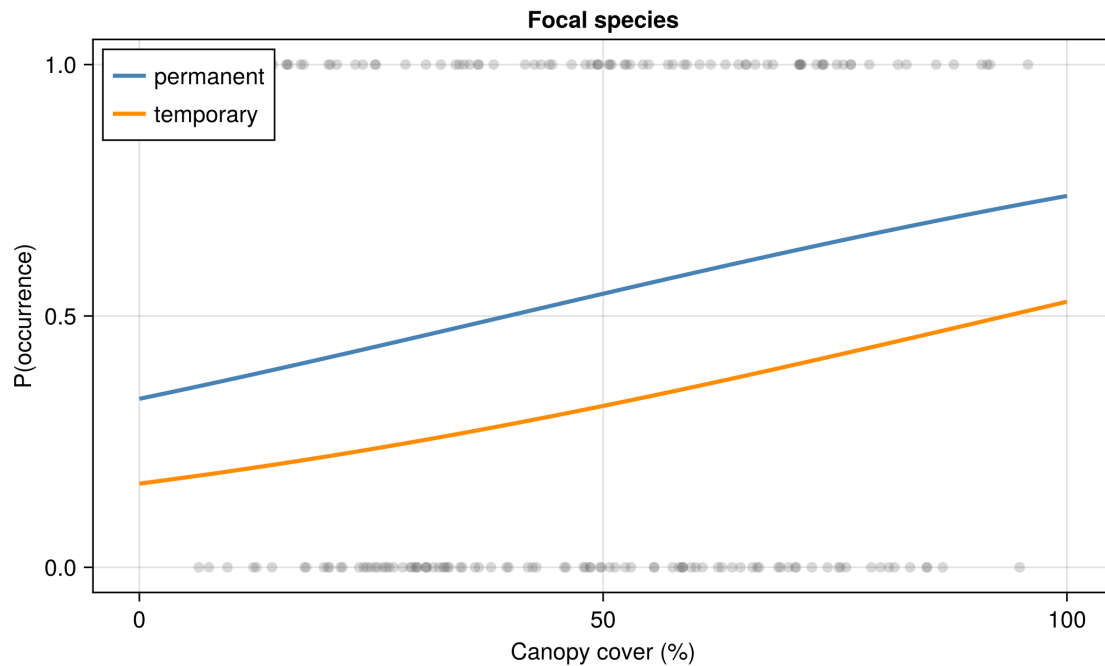


Figure 9.1: Predicted probability of focal-species occurrence along the canopy gradient.

i Your turn 9.5 — Predicting a new pond

A temporary pond has 80% canopy and water at 26 °C. According to `g4`, what is the probability of finding the focal species there? Pass it to `check("9.5", ...)`.

Hint

Build a one-row `DataFrame` with the columns `canopy`, `temp_c` and `hydro`, and pass it to `predict(g4, ...)`, as with `grid` above.

Solution

```
new_pond = DataFrame(canopy = [80.0], temp_c = [26.0],
  ↪ hydro = ["temporary"])
p_new = predict(g4, new_pond)
```

```
1-element Vector{Union{Missing, Float64}}:
 0.5246412509540385
```

```
check("9.5", p_new)
```

✓ Correct!

About 0.52. `predict` returns probabilities for a logistic GLM by default: the link function is undone for you.

9.6 GLMMs

```
gm1 = fit(MixedModel,  
          @formula(richness ~ log_area + canopy + (1 | region)),  
          ponds, Poisson())
```

Minimizing 2 Time: 0:00:00 (0.14 s/it)
 objective: 217.7344900669133

Minimizing 323 Time: 0:00:00 (1.55 ms/it)

| | Est. | SE | z | p | σ_{region} |
|-------------|---------|--------|-------|--------|--------------------------|
| (Intercept) | 0.5192 | 0.2688 | 1.93 | 0.0534 | 0.3534 |
| log_area | 0.3703 | 0.0307 | 12.05 | <1e-32 | |
| canopy | -0.0103 | 0.0013 | -7.72 | <1e-13 | |

In R

```
glmer(richness ~ log_area + canopy + (1 | region),  
      data = ponds, family = poisson)
```

The formula notation is the same, (1 | group) included. For the estimation method used and the options available, consult the `MixedModels.jl` documentation (Bates et al. 2026): these are details that change between versions, and not worth reproducing here from memory.

i Your turn 9.6 — A GLMM with hydroperiod

Add hydro to gm1 and pass the fixed effects of the new model to check("9.6", ...). Compare the hydro estimate with the one from 9.1.

Hint

Add + hydro before the random-effect term. `fixef(gm)` returns the fixed effects.

Solution

```
gm_hydro = fit(MixedModel,  
               @formula(richness ~ log_area + canopy +  
                        ↪ hydro + (1 | region)),  
               ponds, Poisson(); progress = false)  
fixef(gm_hydro)
```

4-element Vector{Float64}:

```
0.6010313133765797
0.3694372355954096
-0.010471771241828185
-0.15987416060204804
```

```
check("9.6", fixef(gm_hydro))
```

✓ Correct!

9.7 Gaps you will hit

This is the point where being honest is worth more than being enthusiastic. The table below is an assessment as of **August 2026**, made by checking the public package registries; check before concluding that something does not exist.

| You need | In R | In Julia today |
|------------------------------|---------------------|------------------------------|
| Zero-inflation, hurdle | glmmTMB, psc1 | no mature equivalent |
| Beta regression | betareg, glmmTMB | no mature equivalent |
| Additive models (GAM) | mgcv | no mature equivalent |
| Simulated quantile residuals | DHARMA | none; implementable by hand |
| Marginal means, contrasts | emmeans | Effects.jl, partial coverage |
| Ordinal regression | ordinal, MASS::polr | partial coverage |

9.7.1 The pragmatic escape hatch: RCall.jl

There is no shame in calling R from inside Julia when the right package only exists there. The workflow stays reproducible, and data crosses the boundary automatically.

```
using RCall

@rput ponds          # send the DataFrame to R

R"""
library(mgcv)
m <- gam(richness ~ s(canopy) + s(log_area) + region,
        family = poisson, data = ponds)
summary(m)
"""
```

i Where this leaves you

For the repertoire covered by this chapter — Poisson, negative binomial, binomial, gamma, offsets, GLMMs — Julia covers what you need, and this chapter has demonstrated it with executed code. Outside it, `RCall.jl` is the bridge. Naming the boundary clearly serves the reader better than hiding it.

Challenges

1. Compute the dispersion statistic for `g2`. Did the negative binomial fix the overdispersion? After 9.3, would you still choose it?
2. Build the predicted-probability curve of Figure 9.1 for each hydroperiod at three temperatures (22, 24.5 and 27 °C). How much does temperature shift the curve?
3. Implement DHARMA-style quantile residuals for `g1`: simulate 250 datasets from the fitted model and, for each pond, compute the proportion of simulated values below the observed one. If the model is right, these proportions are uniform. Are they? Repeat for the model from 9.3.

Part IV

Programming for ecology

10 Writing your own functions

Sooner or later every analysis needs a calculation that no package provides — an index from a paper, a data check specific to your field protocol, a summary your lab always reports. This chapter is about turning that calculation into a function that is easy to call, hard to misuse, documented and tested.

Section 4.3 introduced the syntax. Here the focus is on what makes a function good: sensible arguments, clear errors, a docstring, and tests that tell you when you break it.

i Where this chapter comes from

This chapter and Chapter 11 are **inspired by** the programming and resampling modules of **ecoR** (Oliveira 2026), the R course wiki of the Instituto de Biociências, Universidade de São Paulo, produced and maintained by Alexandre Adalardo de Oliveira. The idea of building ecological indices as functions, step by step, comes from there. The text, examples and exercises here are written from scratch for Julia; the authors of ecoR have not reviewed this book.

If you read Portuguese and want to go further, the original material is freely available (CC BY-SA 4.0):

- [9. Noções de Programação](#) — the course notes on programming;
- [9. Criando Funções Básicas](#) — the step-by-step tutorial;
- [exercises on functions](#), automatically graded in [notaR](#), in R.

```
using TidierData
using DataFrames, CSV
using Statistics
using Test

ponds = CSV.read(joinpath("../", "data", "ponds.csv"), DataFrame)
nrow(ponds)
```

180

i Exercises in this chapter

As in Chapter 4, most answers here are functions you write, which check calls on its own test inputs. Start each session with:

```
include("exercises/check.jl")
using TidierData, DataFrames, CSV, Statistics, Test
ponds = CSV.read(joinpath("data", "ponds.csv"), DataFrame)
```

10.1 From a calculation to a function

Suppose you want to know how variable richness is across ponds, relative to its mean. The coefficient of variation answers that, and it is one line:

```
std(ponds.richness) / mean(ponds.richness) * 100
```

68.12190985250986

The moment you want it for a second variable, or per region, the line becomes a function. Give it a name that says what it computes, and the argument a name that says what it expects:

```
cv(x) = std(x) / mean(x) * 100

cv(ponds.richness), cv(ponds.body_size)
```

(68.12190985250986, 9.971208205402203)

Richness varies far more, relative to its mean, than body size does.

A function can return several things at once. The idiomatic way is a **named tuple**, whose fields you read by name:

```
count_summary(x) = (n = length(x), mean = mean(x), var = var(x))

s = count_summary(ponds.richness)
s.mean, s.var
```

(7.133333333333334, 23.613407821229057)

i Your turn 10.1 — Variance-to-mean ratio

For counts, the variance-to-mean ratio says how individuals — or species — are spread across sampling units. If the counts follow a Poisson distribution, variance equals the mean and the ratio is about 1; above 1 the counts are more clumped, below 1 more even. Write `vmr(x)` and check it with `check("10.1", vmr)`.

Hint

Same shape as `cv`: one line, var over mean. var is the sample variance, which divides by $n - 1$.

Solution

```
vmr(x) = var(x) / mean(x)

vmr(ponds.richness)
```

3.310290816060148

```
check("10.1", vmr)
```

✓ Correct!

Richness across all 180 ponds has a ratio of about 3.3: far more variable than a Poisson with a single mean. That does not yet mean the counting process is odd — the ponds differ in area, canopy and region. Section 10.5 returns to this.

10.2 Arguments: defaults and keywords

A function often has a sensible default that the user may want to change. `cv` returns a percentage; someone might prefer a proportion. There are two ways to offer that.

A **positional argument with a default** comes after the others and is set by position:

```
cv_pos(x, percent = true) = std(x) / mean(x) * (percent ? 100 : 1)

cv_pos(ponds.richness), cv_pos(ponds.richness, false)
```

(68.12190985250986, 0.6812190985250985)

A **keyword argument** goes after a semicolon and is always set by name:

```
cv_kw(x; percent = true) = std(x) / mean(x) * (percent ? 100 : 1)

cv_kw(ponds.richness; percent = false)
```

0.6812190985250985

For options like this, keywords are the better choice: `cv_kw(x; percent = false)` says what it does, while `cv_pos(x, false)` makes the reader look up what false means.

A function can also collect keywords it does not know and pass them on with `kwargs...`. Here `rounded_cv` handles `digits` itself and hands everything else to `cv_kw`:

```
rounded_cv(x; digits = 1, kwargs...) = round(cv_kw(x; kwargs...);
↳ digits)

rounded_cv(ponds.richness), rounded_cv(ponds.richness; percent = false,
↳ digits = 3)
```

(68.1, 0.681)

In R

R has one kind of argument, which can be given by position or by name. Julia separates them: arguments before the `;` are positional, those after are keywords and must be named. `kwargs...` plays the role of R's `...` when forwarding options to another function.

Your turn 10.2 — Standardizing, with options

Write `standardize(x; center = true, scale = true)`. By default it returns $(x - \bar{x})/s$; `center = false` skips subtracting the mean and `scale = false` skips dividing by the standard deviation. Check it with `check("10.2", standardize)`.

Hint

Build the result in two steps, each depending on its option: `centered = center ? x .- mean(x) : x`, then divide `centered` by `std(x)` only if `scale` is `true`.

Solution

```
function standardize(x; center = true, scale = true)
    centered = center ? x .- mean(x) : x
    return scale ? centered ./ std(x) : centered
end

check("10.2", standardize)
```

✓ Correct!

`std(x)` is computed on the original `x`, so it is the same whether or not you center.

10.3 Refusing bad input

A function that quietly returns nonsense is worse than one that stops. `cv` has two blind spots:

```
cv([0, 0, 0]), cv([-5, 5])
```

(NaN, Inf)

NaN for an empty pond survey, and an infinite coefficient for data with a negative value, which counts cannot have. Checks at the top of the function catch both, and `throw(ArgumentError(...))` tells the caller that the problem is the input, not the function:

```
function cv_checked(x)
  any(<(0), x) && throw(ArgumentError("values must be non-negative"))
  mean(x) > 0 || throw(ArgumentError("the mean is zero; the CV is
    ↪ undefined"))
  return std(x) / mean(x) * 100
end

try
  cv_checked([-5, 5])
catch e
  e
end
```

`ArgumentError("values must be non-negative")`

The condition `&& action` form reads as “if condition, then action”; condition `|| action` as “unless condition, action”. Both are idiomatic for checks like these.

💡 `@assert` is for your own mistakes

`@assert condition` stops when something that should never happen does. Use it to check your own reasoning inside a function; use `ArgumentError` for input that a user could plausibly get wrong.

i Your turn 10.3 — A safer `vmr`

Rewrite `vmr` so that it throws an `ArgumentError` when it gets fewer than two values, any negative value, or a mean of zero, and still returns the ratio otherwise. Check it with `check("10.3", vmr)`.

Hint

Three checks, one line each, before the calculation. `length(x) < 2`, `any(<(0), x)` and `mean(x) == 0` are the conditions.

Solution

```

function vmr(x)
  length(x) < 2 && throw(ArgumentError("need at least
    ↪ two values"))
  any(<(0), x) && throw(ArgumentError("counts cannot be
    ↪ negative"))
  mean(x) == 0 && throw(ArgumentError("the mean is
    ↪ zero; the ratio is undefined"))
  return var(x) / mean(x)
end

check("10.3", vmr)

```

✓ Correct!

This replaces the one-line `vmr` from 10.1: a method with the same argument types overwrites the old one.

10.4 Documenting a function

A string placed immediately before a function becomes its **docstring**, which is what `?cv_documented` shows in the REPL and what VS Code shows when you hover over the name. The convention is the signature indented by four spaces, then one or two sentences on what the function does and returns:

```

"""
    cv_documented(x; percent = true)

Coefficient of variation of `x`: the standard deviation divided by the
    ↪ mean,
times 100 unless `percent = false`. Throws an `ArgumentError` if the
    ↪ mean is zero.
"""
function cv_documented(x; percent = true)
  mean(x) > 0 || throw(ArgumentError("the mean is zero; the CV is
    ↪ undefined"))
  return std(x) / mean(x) * (percent ? 100 : 1)
end

```

Main.Notebook.cv_documented

In R

Docstrings play the role of roxygen2 comments, with one difference: they are part of the language, available as soon as the function is defined, with no build step.

i Your turn 10.4 — Document vmr

Add a docstring to your `vmr` from 10.3: the signature, what the ratio means, and which inputs it refuses. Run the definition again and try `?vmr` in the REPL. Then check it with `check("10.4", vmr)`.

Hint

The docstring goes directly above function `vmr(x)`, with nothing in between — not even a blank line.

Solution

```
"""
    vmr(x)

Variance-to-mean ratio of the counts `x`. About 1 for
  ↳ Poisson counts, above 1 when
counts are clumped, below 1 when they are more even.
  ↳ Throws an `ArgumentError` for
fewer than two values, negative values, or a zero mean.
"""
function vmr(x)
    length(x) < 2 && throw(ArgumentError("need at least
  ↳ two values"))
    any(<(0), x) && throw(ArgumentError("counts cannot be
  ↳ negative"))
    mean(x) == 0 && throw(ArgumentError("the mean is
  ↳ zero; the ratio is undefined"))
    return var(x) / mean(x)
end

check("10.4", vmr)
```

✓ Correct!

10.5 Your function, per group

A function you write works inside a Tidier pipeline — with one catch. Tidier knows that `mean`, `std` or `sum` summarize a whole column, but it does not know that about your `cv`. By default it applies unknown functions to **each value separately**, which here would compute a coefficient of variation for every single pond:

```
wrong = @chain ponds begin
    @group_by(region)
    @summarize(cv_richness = cv(richness))
end
```

```
nrow(wrong), first(wrong.cv_richness, 3))
```

(180, [NaN, NaN, NaN])

180 rows of NaN instead of three numbers, and no error. Putting ~ in front of the function tells Tidier to pass it the whole column:

```
@chain ponds begin
  @group_by(region)
  @summarize(cv_richness = ~cv(richness), cv_body = ~cv(body_size))
end
```

| | region | cv_richness | cv_body |
|---|----------|-------------|---------|
| | String15 | Float64 | Float64 |
| 1 | Atlantic | 56.2043 | 8.88778 |
| 2 | Amazon | 53.2696 | 11.3731 |
| 3 | Cerrado | 60.8118 | 9.92402 |

The same with DataFrames.jl directly (Section 5.7), where the question does not arise:

```
combine(groupby(ponds, :region), :richness => cv => :cv_richness)
```

| | region | cv_richness |
|---|----------|-------------|
| | String15 | Float64 |
| 1 | Atlantic | 56.2043 |
| 2 | Amazon | 53.2696 |
| 3 | Cerrado | 60.8118 |

⚠ Check the number of rows

After @summarize, you should have one row per group. If you have one row per observation, Tidier has applied your function to each value: add the ~.

i Your turn 10.5 — Is richness clumped within regions?

Compute the variance-to-mean ratio of richness for each region, in a column called vmr, using your function. Check it with check("10.5", by_region). Are the ratios closer to 1 than the value for all ponds together?

Hint

The ~cv block of this section, with ~vmr in place of ~cv. Without the ~, your vmr from 10.3 receives one pond at a time and rightly refuses it: “need at least two values”.

Solution

```

by_region = @chain ponds begin
  @group_by(region)
  @summarize(vmr = ~vmr(richness))
end

check("10.5", by_region)

```

✓ Correct!

The ratios, from about 1.8 to 3.2, are all lower than for the pooled ponds (3.3): part of the variation came from differences between regions. They are still above 1, because ponds within a region differ in area and canopy. In Section 9.3 a Poisson model that accounts for these predictors had no overdispersion left. A ratio above 1 in raw counts is a reason to look for predictors, not proof that the counts are not Poisson.

10.6 Functions that take functions

In Julia, functions are values: they can be stored in a vector, passed to another function, or written without a name. That makes it easy to apply several summaries at once:

```
[f(ponds.richness) for f in (mean, median, cv, vmr)]
```

```

4-element Vector{Float64}:
 7.133333333333334
 6.0
68.12190985250986
 3.310290816060148

```

An **anonymous function** is written `x -> expression`. `map` applies one to every element of a collection — here, to three columns of the data:

```

map(col -> round(cv(col); digits = 1), eachcol(ponds[:, [:area_m2,
↪   :temp_c, :body_size]]))

```

```

3-element Vector{Float64}:
117.6
 11.6
 10.0

```

Area varies enormously between ponds, temperature very little: the coefficient of variation puts variables with different units on the same scale.

i Your turn 10.6 — Morisita's index

Morisita's index of dispersion compares how often two individuals picked at random fall in the same sampling unit with what a random spread would give:

$$I_d = n \frac{\sum_i x_i(x_i - 1)}{N(N - 1)}$$

where x_i is the count in unit i , n the number of units and $N = \sum_i x_i$. It is about 1 for a random spread and above 1 for a clumped one. Write `morisita(x)`, check it with `check("10.6", morisita)`, and apply it to these counts of tadpole clusters in ten quadrats along a pond margin:

```
quadrats = [0, 3, 0, 0, 7, 1, 0, 0, 5, 0]
```

```
10-element Vector{Int64}:
```

```
0
3
0
0
7
1
0
0
5
0
```

Hint

`x .* (x .- 1)` gives $x_i(x_i - 1)$ for every unit at once; `sum` adds them up.

Solution

```
function morisita(x)
    N = sum(x)
    return length(x) * sum(x .* (x .- 1)) / (N * (N - 1))
end

morisita(quadrats)
```

```
2.8333333333333335
```

```
check("10.6", morisita)
```

✓ Correct!

About 2.8: the clusters are strongly clumped, in three of the ten quadrats. A version for real use would refuse $N < 2$, as in 10.3.

10.7 Testing your functions

check has been testing your functions against inputs whose answers are known. You can do the same for your own functions with the Test standard library, and rerun the tests every time you change the code:

```
@testset "cv_checked" begin
  @test cv_checked([10, 10, 10]) == 0          # no variation
  @test cv_checked([1, 2, 3]) ≈ 50             # std 1, mean 2
  @test_throws ArgumentError cv_checked([0, 0])
  @test_throws ArgumentError cv_checked([-1, 1])
end;
```

```
Test Summary: | Pass  Total  Time
cv_checked    |    4      4  0.6s
```

Good tests use cases you can work out by hand, and include the inputs the function should refuse. Type `≈` as `\approx` followed by Tab; it compares floating-point numbers with a tolerance, where `==` would fail over rounding.

💡 Where your functions should live

Once a function is useful beyond one script, move it to a file of its own — say `functions.jl` in your project folder — and load it with `include("functions.jl")` at the top of each analysis. The tests can live in a second file next to it. When the collection grows, the next step is a package, which Pkg can create for you with `Pkg.generate`.

Challenges

1. Write a `@testset` for your `morisita` with cases you can compute by hand: all individuals in one unit, and exactly one individual per unit. What should each give?
2. Add a Matrix method to `vmr` (Section 4.4) that returns one ratio per column, treating each column as a species counted across quadrats (rows).
3. `cv` fails on a column with missing values, such as canopy. Add a keyword `skipmissing = false` that, when true, ignores them. Which default is safer, and why?

11 Resampling and simulation

Every test in Chapter 8 and Chapter 9 leans on a theoretical distribution: the t , the normal, the χ^2 . This chapter builds those distributions from the data instead, by shuffling and resampling, and then turns the idea around: simulating data from a known process to ask how often a study of a given size would find an effect.

Julia is a natural fit. Resampling means running the same calculation thousands of times, and in Julia the plain loop that expresses the idea is also fast (Section 4.7), so there is no need to contort the code to avoid it.

i Where this chapter comes from

Like Chapter 10, this chapter is **inspired by ecoR** (Oliveira 2026), the R course wiki of the Instituto de Biociências, Universidade de São Paulo, produced and maintained by Alexandre Adalardo de Oliveira. The sequence — permutation, then bootstrap, then simulation from known distributions, with a null model for species co-occurrence — follows its resampling module. The text, examples and exercises here are written from scratch for Julia; the authors of ecoR have not reviewed this book.

If you read Portuguese and want to go further, the original material is freely available (CC BY-SA 4.0):

- [8. Reamostragem e Simulação](#) — the course notes;
- [8. Reamostragem e Simulação](#) — the step-by-step tutorial;
- [exercises on resampling](#), automatically graded in [notaR](#), in R;
- [Simulando Dados](#), a short course on simulating data for linear, mixed and generalized linear models.

```
using TidierData
using DataFrames, CSV
using Statistics, StatsBase, Random, Distributions
using GLM, HypothesisTests
using CairoMakie

CairoMakie.activate!(type = "png")
Random.seed!(2026)

ponds = @chain CSV.read(joinpath("../", "data", "ponds.csv"), DataFrame)
  ↪ begin
    @drop_missing(canopy)
    @mutate(log_area = log(area_m2))
  end

nrow(ponds)
```

i Exercises in this chapter

Most answers here come from random draws, so yours will not match the solution to the last digit, and check accepts anything within the expected random variation. Use at least as many permutations or simulations as the exercise asks for: with too few, the answer is too noisy to pass. Start each session with:

```
include("exercises/check.jl")
using TidierData, DataFrames, CSV, Statistics, StatsBase, Random,
    ↪ Distributions
using GLM, HypothesisTests
ponds = @chain CSV.read(joinpath("data", "ponds.csv"), DataFrame)
    ↪ begin
        @drop_missing(canopy)
        @mutate(log_area = log(area_m2))
    end
```

11.1 Randomness you can reproduce

Three functions do almost all the work in this chapter. `shuffle` reorders a vector; `sample`, from `StatsBase.jl`, draws from it with or without replacement:

```
x = [1, 2, 3, 4, 5]

(shuffled      = shuffle(x),
 without_replace = sample(x, 3; replace = false),
 with_replace   = sample(x, 5))          # replace = true is the
    ↪ default
```

(shuffled = [1, 3, 4, 2, 5], without_replace = [2, 4, 5], with_replace = [2, 5, 2, 5, 4])

Random numbers in Julia come from a **random number generator** (RNG). Setting its seed, as the setup block does with `Random.seed!(2026)`, makes a script produce the same draws each time it runs. For code you want to control more tightly, create a generator of your own and pass it as the first argument:

```
rng = Xoshiro(42)
sample(rng, x, 3)
```

3-element Vector{Int64}:

```
4
3
3
```

⚠ Same seed, different Julia, different numbers

The Julia manual states that a minor release (1.11 to 1.12, say) *may change* the sequence of random numbers generated from a given seed. A seed makes your analysis reproducible on your setup, not across all versions. For exact reproducibility, save the random data, or pin the Julia version and the `Manifest.toml`. This is also why the exercises here accept a range of answers.

11.2 Permutation tests

In Section 8.1 a Welch t test asked whether body size differs between permanent and temporary ponds. A permutation test asks the same question with no distributional assumption, in five steps:

1. Choose a statistic that measures the effect: here, the difference in mean body size between permanent and temporary ponds.
2. Compute it for the real data.
3. State the null hypothesis as a shuffle: if hydroperiod has no effect, the labels “permanent” and “temporary” are interchangeable.
4. Shuffle the labels many times, recomputing the statistic each time. This builds the distribution of the statistic *under the null hypothesis*.
5. The p-value is the proportion of shuffles with a statistic at least as extreme as the observed one.

```
is_perm = ponds.hydro .== "permanent"
mean_diff(y, g) = mean(y[g]) - mean(y[.!g])

observed = mean_diff(ponds.body_size, is_perm)

B = 9_999
null_diffs = zeros(B)
labels = copy(is_perm)
for b in 1:B
    shuffle!(labels)                # step 3–4: shuffle,
    ↪ recompute
    null_diffs[b] = mean_diff(ponds.body_size, labels)
end

extreme = count(abs.(null_diffs) .>= abs(observed))
p_perm = (extreme + 1) / (B + 1)

(observed = observed, p_perm = p_perm)
```

```
(observed = -0.8929446693657113, p_perm = 0.1073)
```

Two details. `abs` makes the test two-sided: a difference of -0.9 counts as extreme as one of $+0.9$. And the `+ 1` in numerator and denominator counts the observed data as

one of the possible arrangements, which keeps the p-value from ever being exactly zero (Phipson and Smyth 2010).

```
fig = Figure(size = (700, 380))
ax = Axis(fig[1, 1], xlabel = "Difference in mean body size (mm),
    ↪ permanent - temporary",
    ylabel = "Shuffles")
hist!(ax, null_diffs, bins = 50, color = (:grey60, 0.8))
vlines!(ax, [observed, -observed], color = :darkorange, linestyle =
    ↪ :dash, linewidth = 2)
fig
```

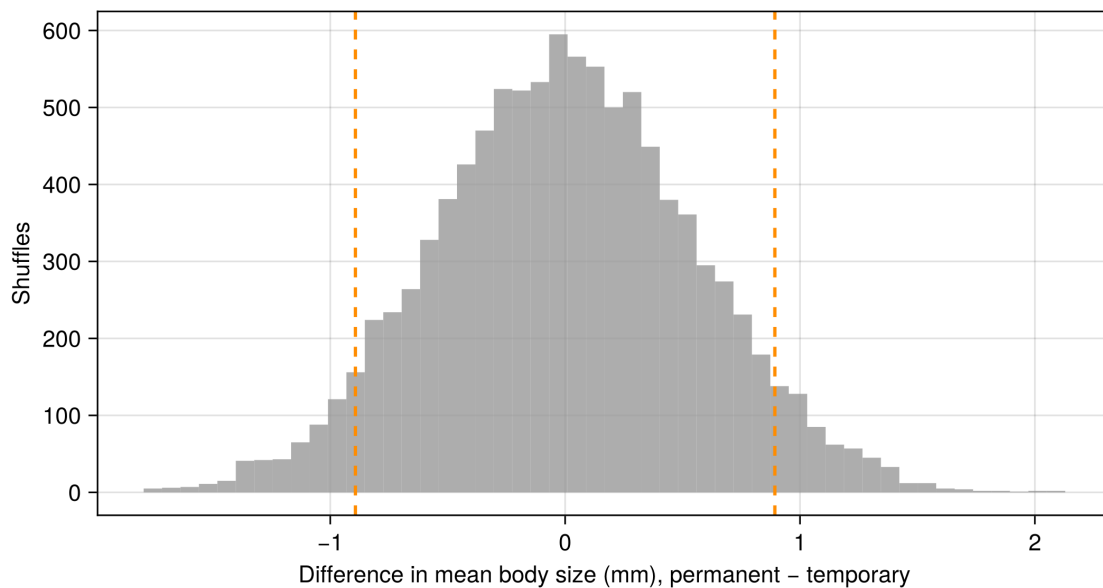


Figure 11.1: Null distribution of the difference in mean body size, from 9 999 shuffles of the hydroperiod labels. The dashed lines mark the observed difference and its mirror image.

The p-value, about 0.10, is close to the 0.103 of the Welch test: with 173 ponds and roughly symmetric data, the theory and the shuffling agree. That will not always be true with small samples or skewed data, which is where permutation earns its keep. HypothesisTests.jl packages the same idea:

```
perm_ponds = ponds.body_size[is_perm]
temp_ponds = ponds.body_size[.!is_perm]
pvalue(ApproximatePermutationTest(perm_ponds, temp_ponds, mean, 9_999))
```

0.10461046104610461

i Your turn 11.1 — Canopy and the focal species

Is canopy cover different in ponds where the focal species occurs? Run a two-sided permutation test on the difference in mean canopy between ponds with `focal == 1` and `focal == 0`, with 9 999 shuffles, and pass the p-value to `check("11.1", ...)`.

Hint

Reuse `mean_diff` and the loop above, with `ponds.canopy` as the response and `ponds.focal == 1` as the groups.

Solution

```
has_focal = ponds.focal == 1
obs_canopy = mean_diff(ponds.canopy, has_focal)

groups = copy(has_focal)
null_canopy = [mean_diff(ponds.canopy, shuffle!(groups))
  ↪ for _ in 1:9_999]
p_canopy = (count(abs.(null_canopy) .>= abs(obs_canopy))
  ↪ + 1) / (9_999 + 1)
```

0.0353

```
check("11.1", p_canopy)
```

✓ Correct!

About 0.04: ponds with the focal species have about 7 points more canopy. The simulation built exactly that in: occurrence rises with canopy (`scripts/make_data.jl`). The comprehension does the same as the loop, in one line.

11.3 Permuting a regression

The same logic tests a slope. If body size does not depend on canopy, any pairing of body sizes with canopy values is as likely as the observed one, so shuffling the response breaks the relationship while keeping both distributions intact:

```
slope(x, y) = cov(x, y) / var(x)

obs_slope = slope(ponds.canopy, ponds.body_size)

y = copy(ponds.body_size)
null_slopes = [slope(ponds.canopy, shuffle!(y)) for _ in 1:9_999]
```

```
extreme = count(abs.(null_slopes) .>= abs(obs_slope))

(slope = obs_slope, extreme = extreme, p = (extreme + 1) / (9_999 + 1))
```

```
(slope = -0.049665628867203854, extreme = 0, p = 0.0001)
```

$\text{cov}(x, y) / \text{var}(x)$ is the least-squares slope, the same number `lm` estimates in Section 8.2. Almost no shuffle produces a slope as steep as the observed one — the count is in `extreme` — so the p-value is at or near the smallest that 9 999 shuffles can show, 1/10 000. To say more you would need more shuffles, but “about 1 in 10 000 or less” is usually all a result needs.

i Your turn 11.2 — Body size and pond area

Test by permutation whether body size is related to pond area: use the slope of `body_size` on `log_area`, two-sided, with 9 999 shuffles. Pass the p-value to `check("11.2", ...)`.

Hint

The block above with `ponds.log_area` in place of `ponds.canopy`. Shuffle a copy of `body_size`, not the column itself, so the data stay intact.

Solution

```
obs_area = slope(ponds.log_area, ponds.body_size)
y = copy(ponds.body_size)
null_area = [slope(ponds.log_area, shuffle!(y)) for _ in
  ↪ 1:9_999]
p_area = (count(abs.(null_area) .>= abs(obs_area)) + 1)
  ↪ / (9_999 + 1)
```

```
0.2583
```

```
check("11.2", p_area)
```

✓ Correct!

About 0.26. The simulation has no effect of area on body size, and the test finds none — correctly, this time. One test in twenty would find one anyway.

11.4 The bootstrap

A permutation test asks what the data would look like if there were no effect. The **bootstrap** (Efron 1979) asks a different question: how much would an estimate change

if we had collected another sample? We cannot collect it, but we can imitate it by drawing a new sample of the same size *from the data themselves, with replacement*. Some ponds appear twice, others not at all, and the spread of the estimate across many such samples measures its uncertainty.

```
n = nrow(ponds)
boot_medians = [median(sample(ponds.body_size, n)) for _ in 1:9_999]

(median = median(ponds.body_size),
 ci_95 = quantile(boot_medians, [0.025, 0.975]))
```

```
(median = 35.56, ci_95 = [35.08, 36.54])
```

The 2.5% and 97.5% quantiles of the bootstrap medians give a 95% **percentile interval** — for a statistic, the median, that has no simple formula for its standard error.

The bootstrap also checks a model's own uncertainty. Resample whole ponds (rows), refit the Poisson model `g1` of Section 9.2 each time, and compare the spread of the canopy coefficient with the standard error the model reports:

```
g1 = glm(@formula(richness ~ log_area + canopy + temp_c), ponds,
  ↪ Poisson(), LogLink())

boot_canopy = map(1:2_000) do _
  rows = sample(1:n, n)
  coef(glm(@formula(richness ~ log_area + canopy + temp_c),
    ponds[rows, :], Poisson(), LogLink()))[3]
end

(bootstrap_se = std(boot_canopy),
 model_se     = stderror(g1)[3],
 ratio        = std(boot_canopy) / stderror(g1)[3])
```

```
(bootstrap_se = 0.0018378945669366084, model_se = 0.0013361976263705467, ratio = 1.3754661)
```

The bootstrap standard error is about 1.4 times larger than the one the Poisson model reports — close to $\sqrt{1.95}$, the square root of the dispersion statistic of `g1` in Section 9.3. The model's standard errors assume the variance equals the mean; the bootstrap assumes nothing about it, and sees the extra variation the model leaves out.

💡 do blocks

`map(1:2_000) do _ ... end` is `map(_ -> ..., 1:2_000)` written as a block, for anonymous functions longer than one line. `_` names an argument you do not use.

i Your turn 11.3 — A bootstrap function

Write `boot_means(x, B)`, which returns a vector with the means of B bootstrap samples of x . Check it with `check("11.3", boot_means)`.

Hint

One comprehension: `[mean(sample(x, length(x))) for _ in 1:B]`. Each bootstrap sample has the same size as x and is drawn with replacement.

Solution

```
boot_means(x, B) = [mean(sample(x, length(x))) for _ in  
    ↪ 1:B]  
  
check("11.3", boot_means)
```

✓ Correct!

i Your turn 11.4 — How rich are Amazonian ponds?

Using `boot_means` with 9 999 samples, compute a 95% percentile interval for the mean richness of the Amazon ponds. Pass it as `[lower, upper]` to `check("11.4", ...)`. Compare it with the usual t interval.

Hint

`ponds.richness[ponds.region == "Amazon"]` selects the Amazon ponds. `quantile(v, [0.025, 0.975])` returns both ends at once.

Solution

```
amazon = ponds.richness[ponds.region == "Amazon"]  
ci_boot = quantile(boot_means(amazon, 9_999), [0.025,  
    ↪ 0.975])
```

2-element Vector{Float64}:
 9.478260869565217
12.956521739130435

```
check("11.4", ci_boot)  
confint(OneSampleTTest(amazon))
```

✓ Correct!

(9.384117221613428, 12.963708865343095)

The two intervals nearly coincide, from about 9.4–9.5 to 13. The bootstrap one starts a little higher: richness is a skewed count, and the bootstrap follows the skew where the t interval is symmetric by construction.

11.5 How many ponds do I need?

Resampling reuses the data you have. Simulation creates data you do not have yet, from a process you specify — and because you chose the process, you know the right answer. That is the logic of this whole book (Section 5.1), and it answers a question every field project faces before it starts: **how many sampling units are enough?**

Suppose permanent ponds hold about 25% more species than temporary ones, as in the simulation behind `ponds.csv` ($e^{0.22} \approx 1.25$). This function generates a survey of n ponds from that process:

```
function simulate_ponds(n; p_temporary = 0.5, rng =
  ↪ Random.default_rng())
  log_area = rand(rng, Normal(5.0, 0.9), n)
  hydro    = ifelse.(rand(rng, n) .< p_temporary, "temporary",
  ↪ "permanent")
  mu       = @. exp(0.9 + 0.28 * log_area + 0.22 * (hydro ==
  ↪ "permanent"))
  richness = rand.(rng, Poisson.(mu))
  return DataFrame(; log_area, hydro, richness)
end

first(simulate_ponds(5), 5)
```

| | log_area | hydro | richness |
|---|----------|-----------|----------|
| | Float64 | String | Int64 |
| 1 | 4.34568 | temporary | 13 |
| 2 | 6.30406 | permanent | 18 |
| 3 | 5.49257 | temporary | 12 |
| 4 | 4.17274 | permanent | 7 |
| 5 | 4.56875 | permanent | 9 |

For a survey of n ponds, fit the model and record whether it detects the effect of hydroperiod at the 5% level. The proportion of simulated surveys that do is the **power** of that design:

```
function detects_hydro(n; kwargs...)
  d = simulate_ponds(n; kwargs...)
  m = glm(@formula(richness ~ log_area + hydro), d, Poisson(),
  ↪ LogLink())
  return coeftable(m).cols[4][3] < 0.05      # p-value of the hydro
  ↪ coefficient
end
```

```

sizes = [20, 40, 60, 80, 100, 140, 180]
power = [mean(detects_hydro(n) for _ in 1:500) for n in sizes]

DataFrame(n = sizes, power = power)

```

| | n | power |
|---|-------|---------|
| | Int64 | Float64 |
| 1 | 20 | 0.35 |
| 2 | 40 | 0.616 |
| 3 | 60 | 0.796 |
| 4 | 80 | 0.886 |
| 5 | 100 | 0.954 |
| 6 | 140 | 0.992 |
| 7 | 180 | 1.0 |

```

fig = Figure(size = (700, 380))
ax = Axis(fig[1, 1], xlabel = "Number of ponds", ylabel = "Power")
lines!(ax, sizes, power, color = :steelblue, linewidth = 2.5)
scatter!(ax, sizes, power, color = :steelblue)
hlines!(ax, 0.8, color = :grey40, linestyle = :dash)
ylims!(ax, 0, 1)
fig

```

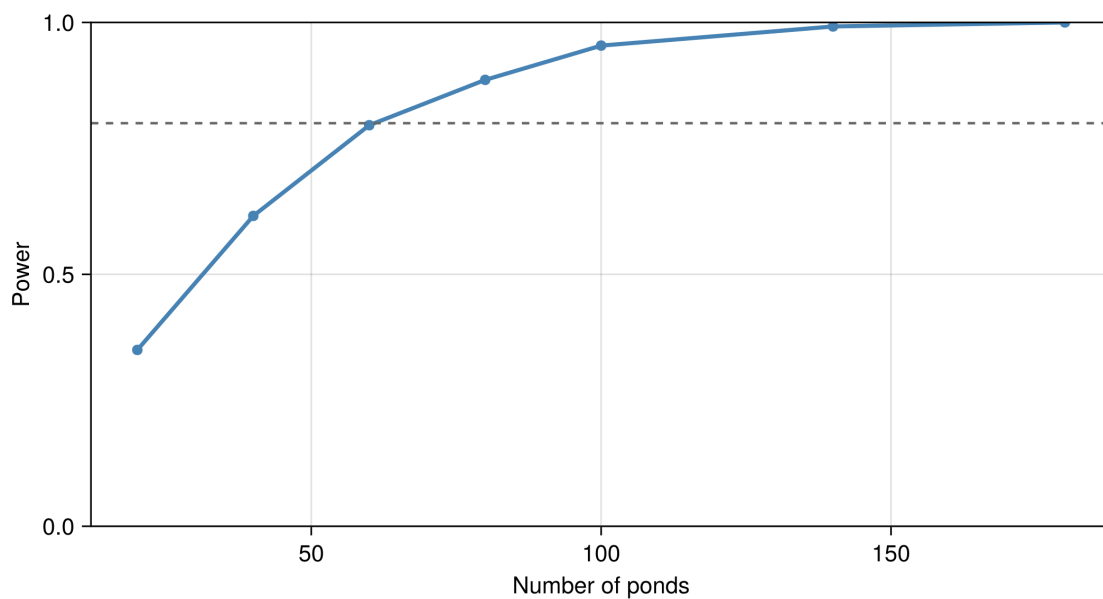


Figure 11.2: Power to detect a 25% difference in richness between permanent and temporary ponds, from 500 simulated surveys per sample size. The dashed line marks the conventional target of 80%.

About 60 ponds reach the usual 80% power; with 20, two surveys in three would miss

an effect that is really there. And a non-significant result from a small survey says little: most of them are non-significant whether or not the effect exists.

i Your turn 11.5 — When the design is unbalanced

In the field you rarely get equal numbers of each kind of pond. Estimate the power with 60 ponds when only 20% are temporary (`p_temporary = 0.2`), from at least 1 000 simulated surveys, and pass it to `check("11.5", ...)`.

Hint

`detects_hydro` passes its keywords on to `simulate_ponds` (Section 10.2), so `detects_hydro(60; p_temporary = 0.2)` already does what you need.

Solution

```
power_unbalanced = mean(detects_hydro(60; p_temporary =  
  ↪ 0.2) for _ in 1:2_000)
```

0.5885

```
check("11.5", power_unbalanced)
```

✓ Correct!

About 0.60, down from 0.80 with a balanced design: the same 60 ponds tell you less, because the comparison rests on only about 12 temporary ones.

11.6 A null model for species co-occurrence

Permutation is also the basis of **null models** in community ecology. The question here is whether species co-occur less than expected by chance, which could indicate competition or different habitat requirements. The data are a presence-absence matrix, one row per species and one column per site.

`ponds.csv` has no species lists, so this section uses a small matrix simulated for the purpose: six species in twelve ponds.

```
M = [1 1 1 0 1 1 0 0 0 0 0 1;  
     1 0 1 1 1 1 0 1 0 0 0 0;  
     0 1 1 1 1 1 0 0 0 1 0 0;  
     1 0 0 0 0 0 1 1 1 0 1 1;  
     0 0 1 0 0 0 1 1 0 1 1 1;  
     0 1 0 0 0 0 1 0 1 1 1 1]
```

```
(species_occurrences = vec(sum(M, dims = 2)),
 pond_richness       = vec(sum(M, dims = 1)))
```

```
(species_occurrences = [6, 6, 6, 6, 6, 6], pond_richness = [3, 3, 4, 2, 3, 3, 3, 3, 2, 3, 3])
```

A common statistic is the **C-score** (Stone and Roberts 1990). For each pair of species, count the “checkerboard units”: with r_i and r_j the number of sites each occupies and S the number of sites they share, the pair contributes $(r_i - S)(r_j - S)$. The C-score is the mean over all pairs. The less species share sites, the higher it is.

i Your turn 11.6 — The C-score

Write `cscore(M)` for a matrix with species in rows and sites in columns, and check it with `check("11.6", cscore)`. The rest of this section uses it.

Hint

`r = vec(sum(M, dims = 2))` gives the occurrences of each species. Loop over pairs with `for i in 1:s-1, j in i+1:s`, where `s = size(M, 1)`; the shared sites are `sum(M[i, :] .& M[j, :])`.

Solution

```
function cscore(M)
  r = vec(sum(M, dims = 2))
  s = size(M, 1)
  total, pairs = 0.0, 0
  for i in 1:s-1, j in i+1:s
    shared = sum(M[i, :] .& M[j, :])
    total += (r[i] - shared) * (r[j] - shared)
    pairs += 1
  end
  return total / pairs
end

cscore(M)
```

```
13.866666666666667
```

```
check("11.6", cscore)
```

✓ Correct!

Is a C-score of about 13.9 high? It depends on what random assemblages *of these species in these ponds* would give. Common species share more sites than rare ones by chance alone, and rich ponds hold more species than poor ones, so a fair null model keeps both the number of occurrences of each species and the richness of each pond fixed, and randomizes only which species go where (Gotelli 2000).

The **swap** algorithm does exactly that. It picks two species and two ponds at random; if they form a checkerboard — $\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ or $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ — it flips it. Rows and column totals never change:

```
function swap!(M; attempts = 1_000)
    nsp, nsites = size(M)
    for _ in 1:attempts
        i, j = sample(1:nsp, 2; replace = false)
        k, l = sample(1:nsites, 2; replace = false)
        if M[i, k] == M[j, l] && M[i, l] == M[j, k] && M[i, k] != M[i,
            ↪ l]
            M[i, k], M[i, l] = M[i, l], M[i, k]
            M[j, k], M[j, l] = M[j, l], M[j, k]
        end
    end
    return M
end

null_c = [cscore(swap!(copy(M))) for _ in 1:999]

obs_c = cscore(M)
(observed = obs_c,
 null_mean = mean(null_c),
 p = (count(null_c .>= obs_c) + 1) / (999 + 1),
 ses = (obs_c - mean(null_c)) / std(null_c))
```

(observed = 13.866666666666667, null_mean = 12.801067734401064, p = 0.011, ses = 3.3547575)

```
fig = Figure(size = (700, 380))
ax = Axis(fig[1, 1], xlabel = "C-score", ylabel = "Null assemblages")
hist!(ax, null_c, bins = 25, color = (:grey60, 0.8))
vlines!(ax, obs_c, color = :darkorange, linestyle = :dash, linewidth =
    ↪ 2)
fig
```

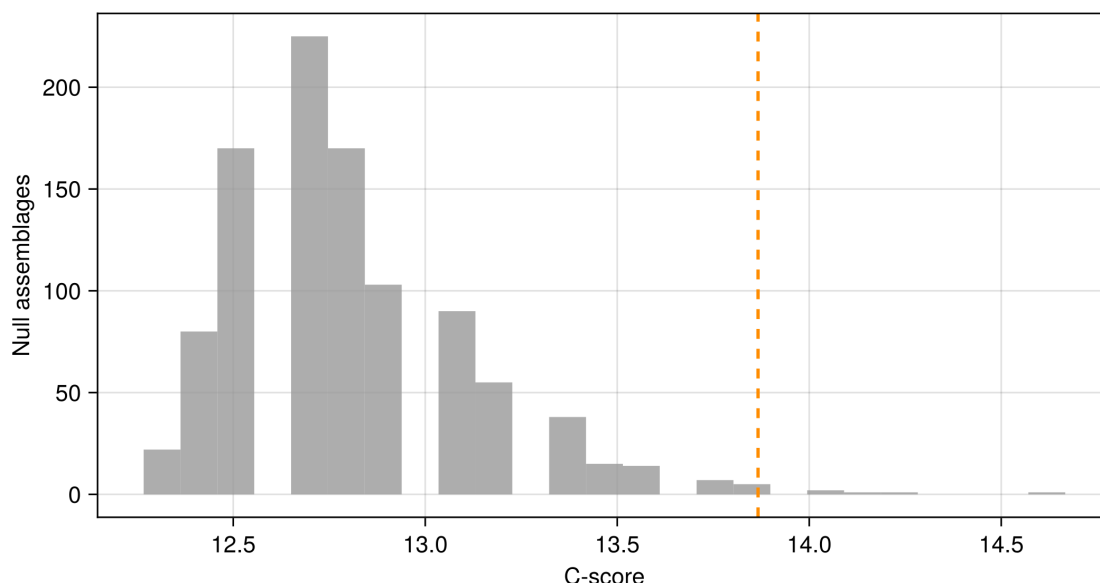


Figure 11.3: C-scores of 999 null assemblages with the same species frequencies and pond richness as the observed one. The dashed line marks the observed C-score.

The observed C-score sits in the upper tail: about 1% of null assemblages are as segregated. The test is one-sided, because the question was about *less* co-occurrence than expected. The **standardized effect size** (ses), the distance from the null mean in null standard deviations, is the usual way to compare such results between matrices of different sizes.

💡 Doing this for real

This is a teaching version. For a real analysis, R's `vegan` package (`nullmodel`, `oecosimu`) implements many null algorithms with the care they need — including how many swaps are enough for the null matrices to be independent, which this version glosses over. Section 9.7.1 shows how to call it from Julia.

Challenges

1. Repeat the permutation test of Section 11.2 with median instead of mean as the statistic. Which would you report for body size, and why?
2. Rerun the power analysis for a 10% effect ($e^\beta = 1.10$). How many ponds would you need? What would you tell a colleague planning a survey of 40 ponds?
3. Build a matrix in which only *one* pair of species is perfectly segregated and the others are scattered at random. Does the fixed-fixed null model detect it? What does that say about the C-score as a measure of a single pair's interaction?

References

- Anscombe, F. J. 1973. "Graphs in statistical analysis." *The American Statistician* 27 (1): 17–21. <https://doi.org/10.1080/00031305.1973.10478966>.
- Bates, Douglas, Phillip Alday, Dave Kleinschmidt, et al. 2026. *MixedModels.jl: A Julia Package for Fitting (Statistical) Mixed-Effects Models*. V. 5.8.2. Released. <https://github.com/JuliaStats/MixedModels.jl>.
- Bates, Douglas, Martin Mächler, Ben Bolker, and Steve Walker. 2015. "Fitting Linear Mixed-Effects Models Using lme4." *Journal of Statistical Software* 67 (1): 1–48. <https://doi.org/10.18637/jss.v067.i01>.
- Bezanson, Jeff, Alan Edelman, Stefan Karpinski, and Viral B. Shah. 2017. "Julia: A Fresh Approach to Numerical Computing." *SIAM Review* 59 (1): 65–98. <https://doi.org/10.1137/141000671>.
- Chamberlin, T. C. 1890. "The Method of Multiple Working Hypotheses." *Science* ns-15 (366): 92–96. <https://doi.org/10.1126/science.ns-15.366.92>.
- Da Silva, Fernando Rodrigues, Thiago Gonçalves-Souza, Gustavo Brant Paterno, Diogo Borges Provete, and Maurício Humberto Vancine. 2022. *Análises Ecológicas no R*. Nupeea; Canal 6. <https://analises-ecologicas.com/>.
- Danisch, Simon, and Julius Krumbiegel. 2021. "Makie.jl: Flexible high-performance data visualization for Julia." *Journal of Open Source Software* 6 (65): 3349. <https://doi.org/10.21105/joss.03349>.
- Efron, Bradley. 1979. "Bootstrap methods: another look at the jackknife." *The Annals of Statistics* 7 (1). <https://doi.org/10.1214/aos/1176344552>.
- Gotelli, Nicholas J. 2000. "Null model analysis of species co-occurrence patterns." *Ecology* 81 (9): 2606–21. [https://doi.org/10.1890/0012-9658\(2000\)081%5B2606:NMAOSC%5D2.0.CO;2](https://doi.org/10.1890/0012-9658(2000)081%5B2606:NMAOSC%5D2.0.CO;2).
- Hurlbert, Stuart H. 1984. "Pseudoreplication and the Design of Ecological Field Experiments." *Ecological Monographs* 54 (2): 187–211. <https://doi.org/10.2307/1942661>.
- Oliveira, Alexandre Adalardo de. 2026. *ecoR: Curso de Introdução à Linguagem R*. Wiki, Instituto de Biociências, Universidade de São Paulo. <https://ecor.ib.usp.br/>.
- Phipson, Belinda, and Gordon K. Smyth. 2010. "Permutation P-values should never be zero: calculating exact P-values when permutations are randomly drawn." *Statistical Applications in Genetics and Molecular Biology* 9 (1). <https://doi.org/10.2202/1544-6115.1585>.

- Rodrigues, Rodolpho Credo, Érica Hasui, Julia Camara Assis, João Carlos Castro Pena, Renata L. Muylaert, et al. 2019. "ATLANTIC BIRD TRAITS: A Data Set of Bird Morphological Traits from the Atlantic Forests of South America." *Ecology* 100 (6): e02647. <https://doi.org/10.1002/ecy.2647>.
- Stone, Lewi, and Alan Roberts. 1990. "The checkerboard score and species distributions." *Oecologia* 85 (1): 74–79. <https://doi.org/10.1007/BF00317345>.
- Zuur, Alain F., Elena N. Ieno, and Chris S. Elphick. 2010. "A protocol for data exploration to avoid common statistical problems." *Methods in Ecology and Evolution* 1 (1): 3–14. <https://doi.org/10.1111/j.2041-210X.2009.00001.x>.
- Zuur, Alain F., Elena N. Ieno, Neil J. Walker, Anatoly A. Saveliev, and Graham M. Smith. 2009. *Mixed Effects Models and Extensions in Ecology with R*. Statistics for Biology and Health. Springer. <https://doi.org/10.1007/978-0-387-87458-6>.

Change log

Each version of the book is archived on Zenodo with its own DOI (see “How to cite” in the [README](#)). Version numbers follow semantic versioning: a change in the first number means a major reorganization, in the second, new content, and in the third, corrections.

v1.0.0 (2026-10-02)

First citable version.

- Chapters 1 to 11, in English and Portuguese: introduction, from question to model (plan only), prerequisites, Julia basics, data wrangling, visualization, exploratory data analysis, linear models, GLMs, functions, and resampling and simulation.
- Exercises with automatic checking (check).
- PDFs of both editions.